
CMT2310A FIFO and Packet Format Usage Guide

Overview

This document describes the operation principle of CMT2310A FIFO, package format, and the interrupt system. For the convenient of users, the contents of configuration register are corresponding to the input parameters on RFPDK in this document.

The product covered in this document is shown in table below.

Table 1. Product Covered in this Document

Product	Frequency	Modem	Function	Configuration	Package
CMT2310A	113 - 960 MHz	(4)(G)FSK/OOK	Transceiver	Register	QFN24

Users can read *AN237 CMT2310A Quick Start Guide* first to get basic using information of CMT2310A for better understanding of this document.

Table of Contents

1. FIFO Operating Principle	3
1.1 FIFO Related Register	3
1.2 FIFO Operating Mode	5
1.3 FIFO Interrupt Timing	7
1.4 FIFO Application Scenario	8
1.4.1 Application scenario: receive data in RX state	8
1.4.2 Application scenario 2: Fill in the data in advance and enter into TX mode	9
1.4.3 Application scenario 3: After entering the TX, filling data while transmitting	9
1.4.4 Application Scenario 4: Send the same packet or the same group of packets	9
1.4.5 Application scenario 5: Sending out a packet with several times	9
2. Packet Format	10
2.1 Data Mode Configuration	10
2.2 Preamble Configuration	11
2.3 Sync Word Configuration	13
2.4 Overall Package Configuration	15
2.5 Address Configuration	17
2.6 FCS1 Configuration	21
2.7 FCS2 Configuration	22
2.8 CRC Configuration	28
2.9 CODEC Configuration	31
2.10 Wi-sun Data Packet Configuration	34
2.11 Special Configuration for TX Data Packets	36
2.12 Direct Transmission Mode	37
3. GPIO and Interrupt	38
3.1 GPIO Configuration	38
3.2 Interrupt Configuration and Mapping	40
3.3 Antenna TX/RX Switching Control	47
4. Appendix	48
4.1 Appendix 1: Sample code for FIFO read/write	48
4.2 Appendix 2: Sample code for GPIO output interrupt configuration	48
5. Revise History	51
6. Contacts	52

1. FIFO Operating Principle

1.1 FIFO Related Register

The corresponding RFPDK graphical user interface and parameter:

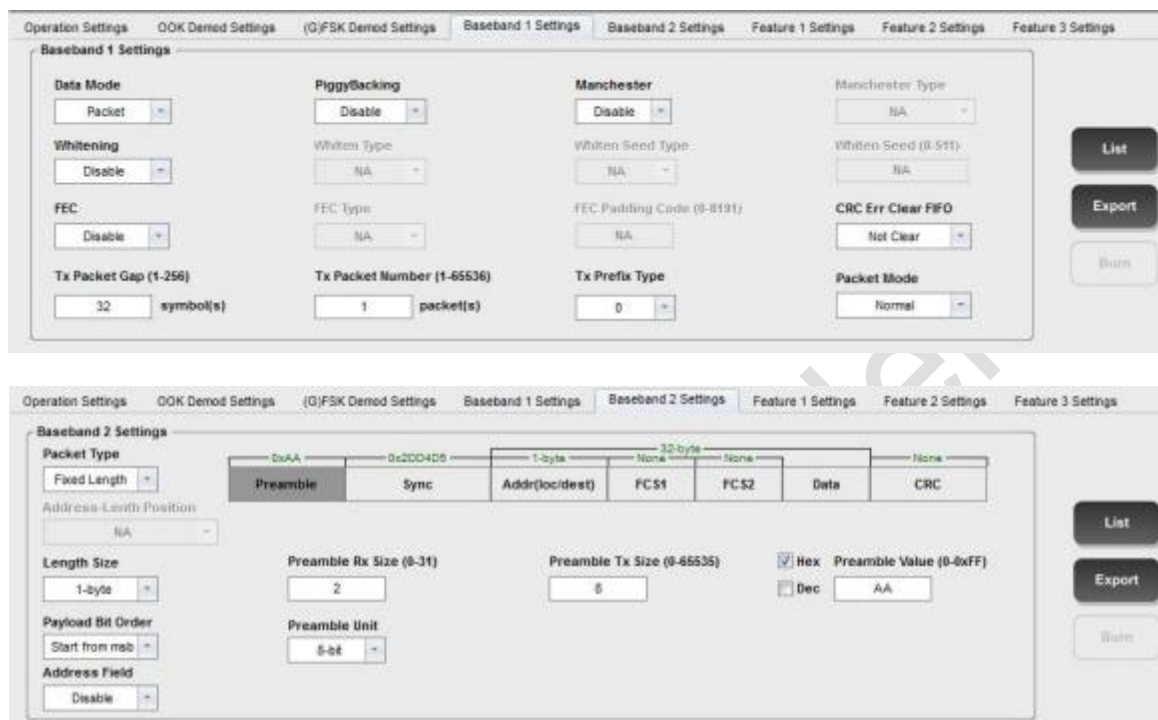


Figure 1. FIFO RFPDK GUI

Table 2. FIFO Related Parameter

Register Bit RFPDK Parameter	Register Bit
Data Mode	DATA_MODE <1:0>
RFPDK do not display and can be configure flexibly by users during application program	FIFO_TH <6:0>
The parameter is automatically calculated based on the TX packet number. When the TX packet number is greater than 1, the parameter is set to 1.	FIFO_AUTO_RES_EN

The register contents and descriptions are shown in the table below.

Table 3. Control Register in Page 0

Register Name	Bits	R/W	Bit Name	Function Description
CTL_REG_19 (0x 13)	6	RW	PD_FIFO	0: save FIFO content in SLEEP state 1: do not save FIFO content in SLEEP state
	5	RW	FIFO_TH<8>	The 8 th bit of FIFO_TH
	4	RW	FIFO_AUTO_CLR_RX_EN	Configure whether to clear RX FIFO before entering into RX mode: 0 : do not clear 1 : clear
	3	RW	FIFO_AUTO_RES_TX_EN	Restore TX FIFO automatically after each packet transmission. If it needs to repeatedly transmit more than 1 packet (TX_PKT_NUM> 0) each time before enters TX, the bit must be set to 1.
	2	RW	FIFO_TX_TEST_EN	0 : TX FIFO can only be write through SPI; 1: TX FIFO can be written through SPI. This bit is only applicable to TX FIFO. It should always be set as 0 except for testing.
	1	RW	FIFO_MERGE_EN	0 : separated as 2 independent 128-byte FIFOs 1 : merge into 1 256-byte FIFO.
	0	RW	FIFO_TX_RX_SEL	When FIFO is in combined mode, 0: FIFO is used as TX FIFO. 1: FIFO is used as RX FIFO.
CTL_REG_20 (0x 14)	7:0	RW	FIFO_TH<7:0>	The FIFO threshold is in Byte unit. For RX, RX_FIFO_TH_FLG will set to 1 if the unread data is under the threshold; For TX, TX_FIFO_TH_FLG will set to 0 if unsent data is under the threshold. If FIFO_MERGE_EN = 0 , the effective range is from 1 to 127 If FIFO_MERGE_EN = 1 , the effective range is from 1 to 255
CTL_REG_27 (0x 1B)	2	W	TX_FIFO_RESTORE	It is used for manually restore TX FIFO by users, restore means reset the read pointer while keep write pointer unchanged, so that TX FIFO will back to unread status and repeatedly transmit the data filled in before.
	1	W	RX_FIFO_CLR	0 : invalid; 1 : clear RX FIFO . After users set the bit to 1, there's no need to set it back to 0 as the system will set it back to 0 internally.

	0	W	TX_FIFO_CLR	0 : invalid; 1 : clear TX FIFO . After users set the bit to 1, there's no need to set it back to 0 as the system will set it back to 0 internally.
CTL_REG_28 (0x1C)	7	R	RX_FIFO_FULL_FLG	Indicate interruption of RX FIFO full 0 : invalid 1 : valid
	6	R	RX_FIFO_NMTY_FLG	Indicates the interruption of a non-empty RX FIFO 0 : invalid 1 : valid
	5	R	RX_FIFO_TH_FLG	Indicates interruption when the RX FIFO unread content exceed FIFO TH 0 : invalid 1 : valid
	3	R	RX_FIFO_OVF_FLG	Indicates interruption of RX FIFO overflow. 0 : invalid 1 : valid
	2	R	TX_FIFO_FULL_FLG	Indicated interruption of TX FIFO full 0 : invalid 1 : valid
	1	R	TX_FIFO_NMTY_FLG	Indicates interruption of a non-empty TX FIFO 0 : invalid 1 : valid
	0	R	TX_FIFO_TH_FLG	Indicates interruption when the TX FIFO unread content exceed FIFO TH 0 : invalid 1 : valid

Table 4. Configuration Register in Page 0

Register Name	Bits	R/W	Bit Name	Function Description
CTL_REG_40 (0x28)	1:0	RW	DATA_MODE<1:0>	Select RX and TX data format: 0 : Direct mode (default) 1 : NA 2 : Packet mode 3 : NA

1.2 FIFO Operating Mode

CMT2310A provides two separated 128-byte FIFO for RX and TX by default, which are mutually irrelevant. Users can also set FIFO_MARGE_EN to 1 to merge the 2 FIFOs into a 256-byte FIFO, which can be used under both TX and RX. It indicates whether it is currently used by TX or RX by configuring the FIFO_RX_TX_SEL.

At usual we recommend that the FIFO working mode has to be preconfigured under STBY state before starting the TX / RX. Registers of CTL_REG_19, CTL_REG_20, CTL_REG_40 can be configurable and saved contents in SLEEP state, so that they only configured once unless you want to change the working mode. CTL_REG_27 and CTL_REG_28 registers are inaccessible under SLEEP state.

Users can choose whether the contents of the FIFO need to be saved in the SLEEP state and configured by the PD_FIFO bit. It further reduces sleep power consumption if setting it as not saving FIFO content in SLEEP mode.

Pre-configuration of the separate FIFO

1. Set DATA_MODE < 1:0> to 2, that is, the data processing mode is Packet Mode.
2. Set FIFO_MERGE_EN to 0
3. Select the subsequently operation:

If it goes to RX:

If users want RX FIFO to be automatically cleared each time when it enters RX, then set FIFO_AUTO_CLR_DIS to 1; otherwise set it to 0 and then manually clear the RX_FIFO_CLR when it is set to 1.

If it goes to TX:

Manually clear the TX_FIFO_CLR when it is set to 1 and then write into the FIFO directly.

Pre-configuration of the merged FIFO

1. Set DATA_MODE < 1:0> to 2, that is, the data processing mode is Packet Mode.
2. Set FIFO_MERGE_EN to 1.
3. Select the subsequently operation:

If it goes to RX:

- a) Set FIFO_TX_RX_SEL to 0 (RX mode) ;
- b) If users need to clear RX FIFO automatically each time when it enters into RX, then set FIFO_AUTO_CLR_RX_EN to 1, otherwise set it to 0 and set RX_FIFO_CLR to 1 to manually clear.

If it goes to TX:

- a) Set FIFO_TX_RX_SEL to 1 (TX mode) ;
- b) Set TX_FIFO_CLR to 1 to manually clear it and write FIFO directly.

When FIFO merges, there is only one FIFO inside the chip for both of the RX / TX is half-duplex and one FIFO is enough. When FIFO is separated into two, it can achieve some special functions for they are relatively independence to each other. For example, if the launch data is the same each time, and RX FIFO can save content under SLEEP mode, then TX FIFO only needs to fill it out once, which can not only save time but also save power consumption. What's more, RX FIFO can constantly write under RX mode, so that users can use the receiving time to fill in the next launched data at the same time,

which will not disturb operation of RX FIFO but also save time in writing in TX FIFO. That's why it can save power consumption.

It should be noted that, every time it switches between TX FIFO and RX FIFO, there should be a manually clearing to guarantee the normal work. In TX FIFO mode, it can only use TX_FIFO_CLR register to conduct clearing while in RX FIFO mode, users can only use the RX_FIFO_CLR register. The two cannot be set to 1 at the same time and users should prevent from confusing.

FIFO_RX_TX_SEL needs to be configured in the FIFO read and write enable operation according to the requirements of the merge operation.

```
cmt2310a_go_ready();
cmt2310a_delay_ms(2);
cmt2310a_clear_interrupt_flag_0(M_TX_DONE_CLR);
/* if fifo merge is enable, enable spi to write the FIFO */
//cmt2310a_fifo_tx_rx_sel(1);
cmt2310a_clear_tx_fifo();

/* The length need be smaller than 128 */
cmt2310a_write_fifo(g_ptr_tx_buffer, g_tx_length);
```

Please refer to Appendix 1 for FIFO read/write operation sample code

1.3 FIFO Interrupt Timing

The interrupt timing diagram of RX FIFO and TX FIFO is shown in the figure below for reference.

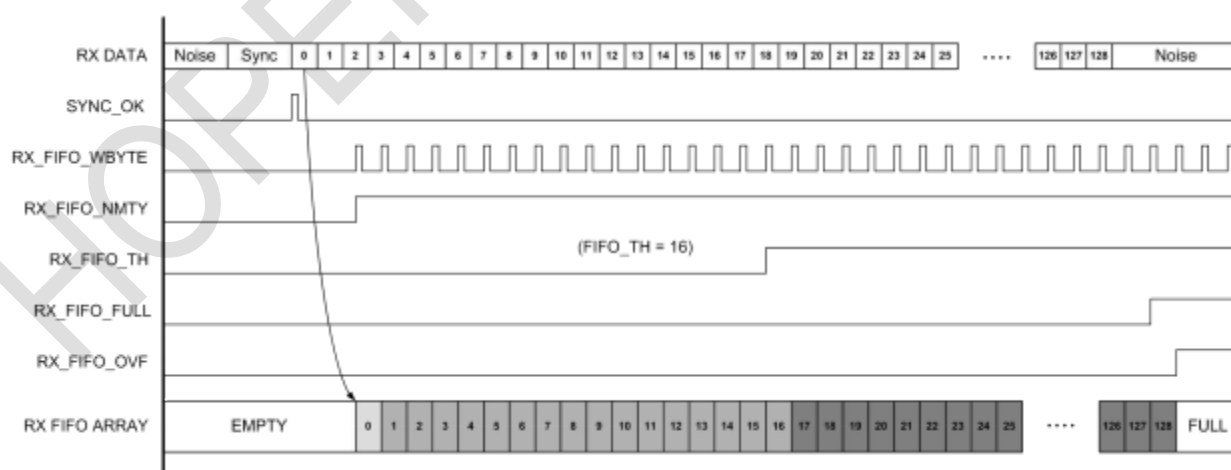


Figure 2. CMT2310A RX FIFO Interrupt Timing Diagram

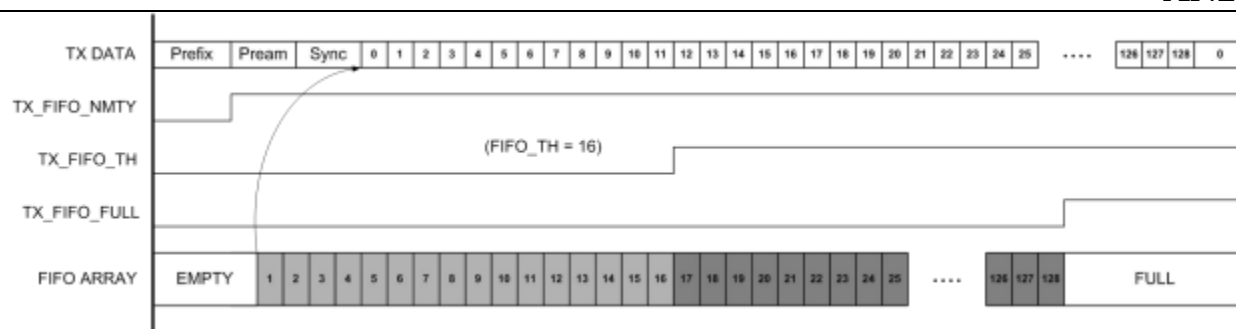


Figure 3. CMT2310A TX FIFO Interrupt Timing Diagram

1.4 FIFO Application Scenario

FIFO starts to work after configuration. The following chapter will introduce some classic application scenarios. To complete the entire TX and RX process, it is a must to configure and control other parts. However, in this chapter, we only introduce related FIFO content.

1.4.1 Application scenario: receive data in RX state

The using of RX FIFO is direct. Every time it enters RX it clears FIFO first and if there is some data received (successfully detect Sync Word), RX FIFO will begin to fill in and MCU can achieve the several operations below according to different interrupts. It does not need extra processing, only need to send RX_FIFO_CLR to clear before the next receiving,

1. Detect the RX_FIFO_FULL interrupt. Once this interrupt is valid it indicates that the FIFO has been fully filled and users can start reading. The packet length is exactly equal to the FIFO depth and users do not read the FIFO until the entire packet is received.
2. Detect the RX_FIFO_TH interrupt. Once this interrupt is valid it indicates that the FIFO has been filled to the preset data length and users can start reading. The packet length is not equal to the FIFO depth and users do not read the FIFO until the entire packet is received.
3. Detect the RX_FIFO_NMTY interrupt. Once this interrupt is valid it needs to read data immediately until the interrupt is invalid. Therefore, it can implement reading while writing. This selection is suitable for packet length greater than the FIFO depth, but also suitable for less than and equal cases.
4. Detect the RX_FIFO_WBYTE interrupt. Once this interrupt is valid, it reads data immediately and fulfill the regular patter of writing one byte after reading one byte at the premise of SPI speeding faster than data receiving.

What's more, users can define how many packets are sent each time by setting the TX_PKT_NUM <7:0>. If the Tx packet number is more than one, it needs to set FIFO_AUTO_RES_EN to 1, that is, let it automatically clear the read pointer of the TX FIFO to make it go back to the unread state after each packet is sent. This allows users to send the same packet repeatedly with no need for MCU to fill in the data again. Users can set the TX_PKT_GAP <7:0> to define the time gap between adjacent packets with a unit of symbol. After entering the transmitting state, the transmitter sends N packets according to these configurations. After completing transmission, the transmitter automatically exits TX and returns to the specified state. The exiting state can be configured by TX_EXIT_STATE <1:0>.

If continuous reception is required, it is recommended to use the fourth operation above. The SPI reads need to be fast enough to be at least 1.5 times faster than a FIFO writing byte. For example, if the data rate is 10 KHz, the time it takes to receive a byte is about 800 us. The rate in which SPI reads a byte is 1.5 times faster than the rate of FIFO writing a byte, taking up to 534 us. Reading a byte through SPI takes 8 SCL clock periods. Adding the time cost at the beginning and end, we can count it as 10 SCL clock cycles, thus each cycle is 53.4 us. After converting, the SCL clock frequency is about 18.7 kHz. Thus roughly figure out that the rate of SCL is 2 times of the data rate.

1.4.2 Application scenario 2: Fill in the data in advance and enter into TX mode

For many applications, it needed to fill the data packets in advance before transmitting into TX FIFO, and then enter the TX. It is suggested that data is filled in STBY state. This scenario is suitable for packet length less than or equal to FIFO depth, and the application time is not so tight. After pre-configuration, users can write in data directly and detect whether the interruption of TX_FIFO_NMTY, TX_FIFO_TH and TX_FIFO_FULL is completely writes in. During the debugging state, users can read back the data to make sure it was written correctly by the following methods.

1. Set FIFO_TX_TEST_EN to 1 to enter the read-back mode of TX FIFO.
2. Read data to confirm whether it is correct.
3. Set FIFO_TX_TEST_EN to 0 to exit the read-back mode of TX FIFO.
4. Set FIFO_CLR_TX to 1 and clear FIFO.
5. Write data again to prepare for transmission.

1.4.3 Application scenario 3: After entering the TX, filling data while transmitting

If TX FIFO is empty when users enter TX state, the chip will keep on transmitting the pre-configured length of Preamble message before it ended and begin transmitting the SyncWord section. After the transmission of SyncWord, FIFO content begins to be read and transmitted. If the FIFO is still not effectively filled, the packet is sent with the FIFO of 0x00. Note that the 0 chip is also considered as the packet content internally, which occupies the valid length of the Payload, and participates in the encoding and CRC calculation. In addition, in the case of super-long packets (the packet content is larger than the FIFO buffer space inside the chip), the SPI speed should be fast enough, and the SCL frequency should be at least twice the data rate. In the process of transmitting data, if the SPI filling speed fails to catch up, that is, the FIFO is read empty by the transmitter, the transmission will not stop. It will keep sending 0 until the FIFO fills in the next data. However, this is an inappropriate operation, which should be avoided as far as possible by users.

1.4.4 Application Scenario 4: Send the same packet or the same group of packets repeatedly each time

For the reason that the TX FIFO will save content in the SLEEP state, so if the data sent each time is the same, the MCU only needs to fill in the data once. And after the TX, it will enter READY and set the FIFO_RESTORE_TX to 1 (there is no need for setting it back to 0 for the bit will clear automatically) with the FIFO read pointer cleared to zero, which means FIFO back to the unread status and save all the content. When the next time entering TX, same content will transmit and re-launch in a new cycle. If FIFO_AUTO_RES_TX_EN has been set to 1, then there is no need to manually set FIFO_RESTORE_TX. Users can directly enter TX to launch.

1.4.5 Application scenario 5: Sending out a packet with several times

The TX FIFO can save not only the content but also pointer state in the SLEEP state. For example, with the combined FIFO size of 256-byte, the user sends four packets of 64-byte each time, users can fill the FIFO of the four packets in order before READY to ensure that the FIFO is full. Then, set the data length to 64(the data length is depending on the packet format, details will show at the following section) and packet number to 1 as well as FIFO_AUTO_RES_TX_EN to 0. After entering TX, the chip will send the first 64-byte packet and exit to READY state after completion, and then re-enter TX, sending the second packet until the fourth sending is completed. In this process, MCU does not need extra operations, but only detects the interruption and switches the state.

2. Packet Format

CMT2310A applies unified configuration of both TX and RX, which is a relatively flexible packet format. The Structure diagram is shown as follow.

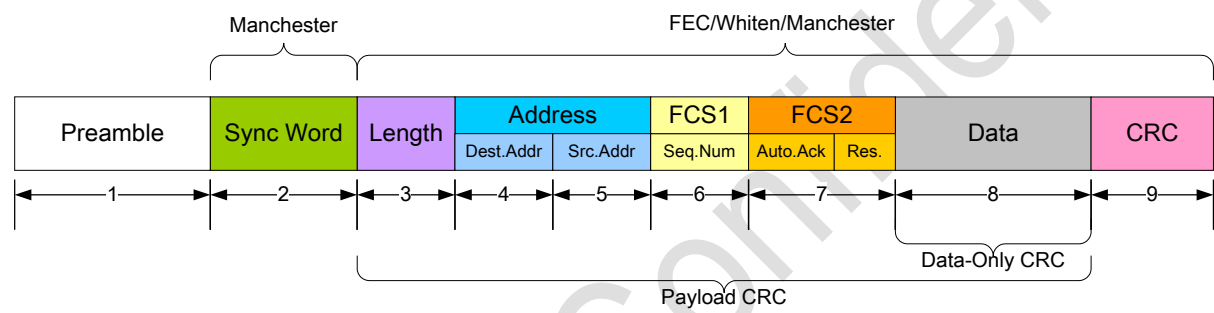


Figure 4. Packet Structure Diagram

The packet format contains 9 optional parts, which can satisfy most single-field structure requirements in the market, while incompatible with multiple Field structures. The configurable parts in the structure are all located in 'base-band zone'. The following will discuss the configuration of various parts along with corresponding register information.

2.1 Data Mode Configuration

Data mode refers to the way through which an external MCU inputs transmit data or received data. The corresponding RFPDK GUI and parameters are shown as follow.



Figure 5. Data Mode Graphical User Interface on RFPDK

Table 5. Data Mode Related Parameters

Register Bit RFPDK Parameter	Register Bit
Data Mode	DATA_MODE < 1:0>
This parameter is automatically selected according to Data Mode. When Data Mode is Direct, it selects data input from GPIO directly; when Data Mode is Packet, it fetches data from TX FIFO.	TX_DATA_MODE

Table 6. Registers in Configuration Area

Register Name	Bits	R/W	Bit Name	Function Description
CTL_REG_40 (Page0, 0x28)	1:0	RW	DATA_MODE< 1:0>	Select TX and RX data mode: 0 : Direct mode (default) 1 : NA 2 : Packet 模式 3 : NA
TX_DR_REG_02 (Page1, 0x17)	7	RW	TX_DATA_MODE	Select TX data source. 0 : TX data input from GPIO directly 1 : TX data fetches from TX FIFO

The two data modes differ in:

- **Direct** - If it is under Direct mode, the RX mode supports preamble and sync detects while FIFO didn't work. TX does not support any packet formats.
- **Packet** - If it is under Packet format mode, it supports all packet format configurations with FIFO in operation.

2.2 Preamble Configuration

The corresponding RFPDK GUI and parameters are as follows.

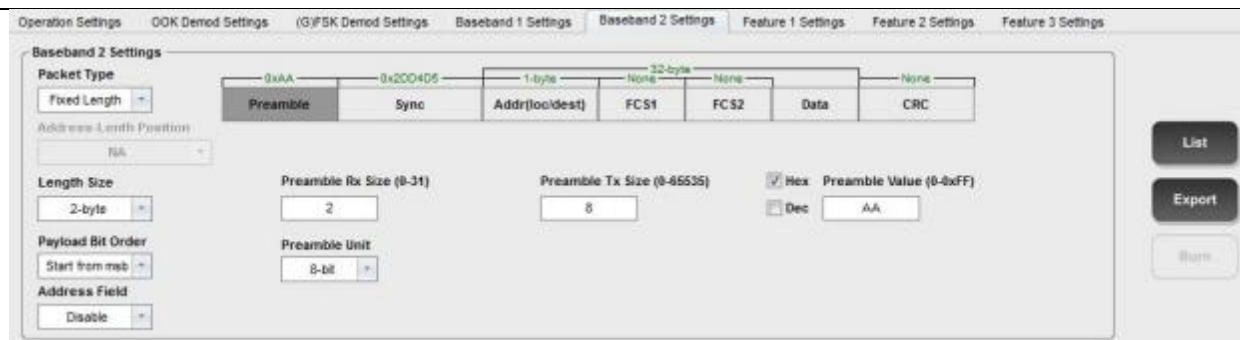


Figure 6. Preamble GUI on RFPDK

Table 7. Preamble Related Parameters

RFPDK Parameter of Register Bit	Register Bit
Preamble Rx Size	RX_PREAM_SIZE<4:0>
Preamble Tx Size	TX_PREAM_SIZE< 15:0>
Preamble Unit	PREAM_UNIT
Preamble Value	PREAM_VALUE<7:0>

Table 8. Registers in Configuration Area

Register Name	Bits	R/W	Bit Name	Function Description
CTL_REG_40 (0x28)	7:3	RW	RX_PREAM_SIZE<4:0>	RX mode Preamble length can be configured from 0 - 31 length units. 0 represents that Preamble is not detected.. 1 represents 1 Preamble length unit is detected, and so on. ..
	2	RW	PREAM_LEN_UNIT	The length unit of Preamble is shared by both TX and RX: 0: the unit is 8 bits. 1: the unit is 4 bits.
CTL_REG_41 (0x29)	7:0	RW	TX_PREAM_SIZE<7:0>	The Preamble length in TX mode can be configured from 0-65535 unit length. 0 represents not sending Preamble,
CTL_REG_42 (0x2A)	7:0	RW	TX_PREAM_SIZE< 15:8>	1 represents sending Preamble with 1 length unit and so on...
CTL_REG_43 (0x2B)	7:0	RW	PREAM_VALUE<7:0>	Preamble value, shared by both TX and RX: When PREAM_LEN_UNIT = 0, 8 bits are valid. When PREAM_LEN_UNIT =1, Only <3:0> are valid

For RX, successful Preamble detection will generate PREAM_OK interrupts and Preamble detection will continue throughout the receiving phase, and it is recommended that users only detect the interrupt when necessary.

2.3 Sync Word Configuration

The corresponding RFPDK GUI and parameters are as follows.

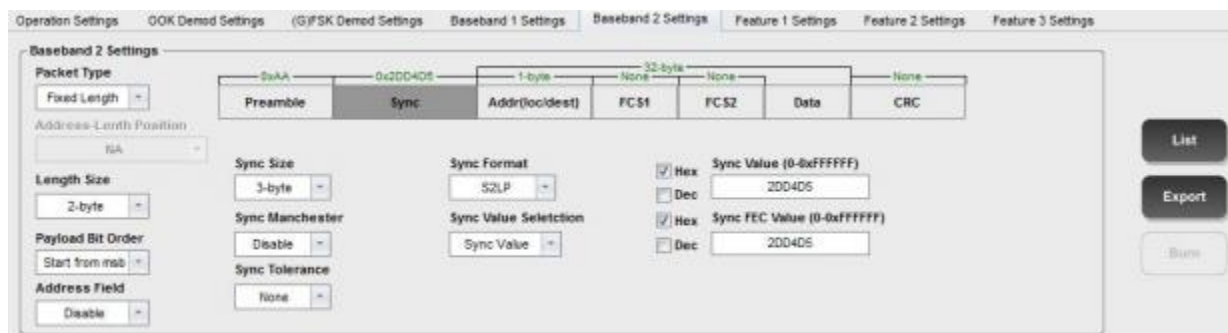


Figure 7. Sync Word GUI on RFPDK

Table 9. Sync Word Related Parameters

Register Bit	RFPDK Parameter	Register Bit
Sync Size		SYNC_SIZE<2:0>
Sync Format		SYNC_MODE_SEL
Sync Value Selection		SYNC_VALUE_SEL
Sync Value		SYNC_VALUE<63:0>
Sync FEC Value		SYNC_FEC_VALUE<63:0>
Sync Tolerance		SYNC_TOL<2:0>
Sync Manchester		SYNC_MAN_EN

Table 10. Registers in Configuration Area

Register Name	Bits	R/W	Bit Name	Function Description
CTL_REG_44	7	RW	SYNC_MODE_SEL	0 : Conventional mode 1 : 802.15.4 mode
	6:4	RW	SYNC_TOL<2:0>	Number of fault-tolerant bits detected by the RX mode to the Sync Word: 0: No error is allowed 1: 1-bit receiving error is allowed. 2: 2-bit receiving error is allowed. 3: 3-bit receiving error is allowed. 4: 4-bit receiving error is allowed. 5: 5-bit receiving error is allowed. 6: 6-bit receiving error is allowed. 7: 7-bit receiving error is allowed.

(0x2C)	3:1	RW	SYNC_SIZE<2:0>	Sync Word Length: 0: 1 byte 1: 2 bytes 2: 3 bytes 3: 4 bytes 4: 5 bytes 5: 6 bytes 6: 7 bytes 7: 8bytes
	0	RW	SYNC_MAN_EN	Sync Word Manchester CODEC Enabling: 0 : Disable 1 : Enable
CTL_REG_45 (0x2D)	7:0	RW	SYNC_VALUE<7:0>	Sync Word value. It is set according to different SYNC_SIZE filled in different registers. More details please see in the table below.
CTL_REG_46 (0x2E)	7:0	RW	SYNC_VALUE< 15:8>	
CTL_REG_47 (0x2F)	7:0	RW	SYNC_VALUE<23: 16>	
CTL_REG_48 (0x30)	7:0	RW	SYNC_VALUE<31:24>	
CTL_REG_49 (0x31)	7:0	RW	SYNC_VALUE<39:32>	
CTL_REG_50 (0x32)	7:0	RW	SYNC_VALUE<47:40>	
CTL_REG_51 (0x33)	7:0	RW	SYNC_VALUE<55:48>	
CTL_REG_52 (0x34)	7:0	RW	SYNC_VALUE<63:56>	
CTL_REG_53 (0x35)	7:0	RW	SYNC_FEC_VALUE<7:0>	The Sync FEC Word value is filled to the SYNC filed when it is FEC_ON at TX mode and when it is detected at RX mode, it will enable FEC. Different registers will be filled according to different SYNC_SIZE. More details please see in the table below.
CTL_REG_54 (0x36)	7:0	RW	SYNC_FEC_VALUE< 15:8>	
CTL_REG_55 (0x37)	7:0	RW	SYNC_FEC_VALUE<23: 16>	
CTL_REG_56 (0x38)	7:0	RW	SYNC_FEC_VALUE<31:24>	
CTL_REG_57 (0x39)	7:0	RW	SYNC_FEC_VALUE<39:32>	
CTL_REG_58 (0x32A)	7:0	RW	SYNC_FEC_VALUE<47:40>	
CTL_REG_59 (0x3B)	7:0	RW	SYNC_FEC_VALUE<55:48>	
CTL_REG_60 (0x3C)	7:0	RW	SYNC_FEC_VALUE<63:56>	

CTL_REG_64 (0x40)	7	RW	SYNC_VALUE_SEL	It is valid when SYNC_MODE_SEL is 0: 0: select SYNC_VALUE 1: select SYNC_FEC_VALUE
----------------------	---	----	----------------	--

	SYNC_VALUE/SYNC_FEC_VALUE							
SYNC_SIZE	<63:56>	<55:48>	<47:40>	<39:32>	<31:24>	<23:16>	<15:8>	<7:0>
0	√							
1	√	√						
2	√	√	√					
3	√	√	√	√				
4	√	√	√	√	√			
5	√	√	√	√	√	√		
6	√	√	√	√	√	√	√	
7	√	√	√	√	√	√	√	√

The tick in the table indicates the register to be filled in. For example, if SYNC_SIZE is set to 1, that is, 2 bytes in length, and the synchronization word is 0x5678, the user fills this value in SYNC_VALUE<63:56> and SYNC_VALUE<55:48>; The MSB corresponds to the sixty-third bit. The LSB corresponds to the forty-eighth bit, namely filling 0x56 into SYNC_VALUE<63:56> and filling 0x78 into SYNC_VALUE<55:48>.

In addition, some applications need to have Manchester encoding for the entire packet, while most of the applications only need to encode Payload. So, it makes a Manchester encoding enabling bit for part of the Sync Word.

2.4 Overall Package Configuration

The corresponding RFPDK GUI and parameters are as follow.

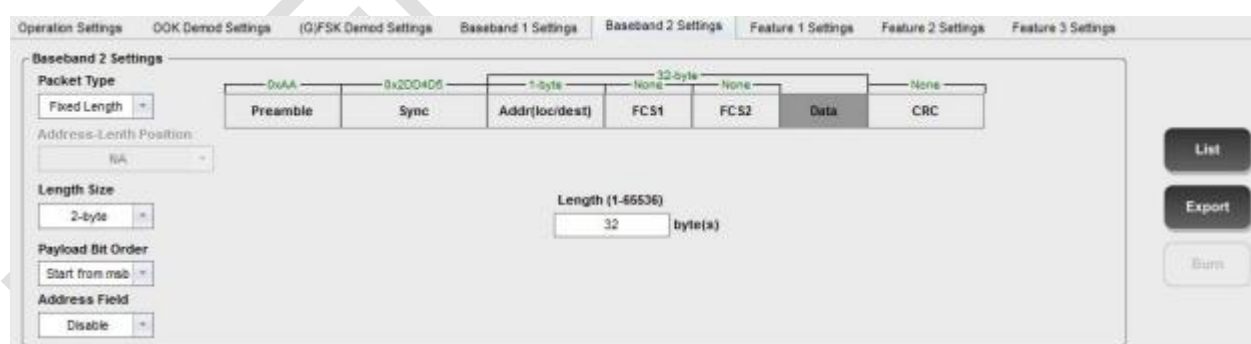


Figure 8. Data Packet GUI on RFPDK

Table 11. Data Packet Related Parameters

Register Bit	RFPDK Parameter	Register Bit
Packet Type		PKT TYPE

Length, which is calculated according to various parameters. More details will be described later.	PAYLOAD_LEN< 15:0 >
Address- Length Position	ADDR_LENGTH_POS_SEL
Payload Bit Order	PAYLOAD_BIT_ORDER
Length Size	LENGTH_SIZE

Table 12. Registers in Configuration Area

Register Name	Bits	R/W	Bit Name	Function Description
CTL_REG_61 (0x3D)	7:0	RW	PAYLOAD_LENGTH<7:0>	Bits <7:0> of the 16-bit Payload length.
CTL_REG_62 (0x3E)	7:0	RW	PAYLOAD_LENGTH< 15:8>	Bits < 15:8> of the 16-bit Payload length.
CTL_REG_63 (0x3F)	5	RW	LENGTH_SIZE	0: 1 byte, supporting variable packet length with 0 - 255 Bytes. 1: 2 bytes, supporting variable packet length with 0 - 65535 Bytes
	2	RW	NODE_LENGTH_POS_SEL	In a packet with a variable length, the location relation between Address and Length Byte. 0: Address is before length Byte 1: Address is after length Byte
	1	RW	PAYLOAD_BIT_ORDER	0: encode and decode starting from MSB of each byte of payload+ CRC 1: encode and decode starting from LSB of each byte of payload+ CRC
	0	RW	PKT_TYPE	Packet length type. 0: fixed packet length. 1: variable packet length.

There's no length field in fixed packet length mode. Packet length is determined by the configuration of PAYLOAD_LENGTH during TX and RX. In variable packet length mode, value of length field is determined according to the PAYLOAD_LENGTH, that is, Length= PAYLOAD_LENGTH, which means the number of payload bytes. The number of received payload s is determined from the value of the received Length. When LENGTH_SIZE is 0, the length field takes the value of PAYLOAD_LEN [7:0]; when LENGTH_SIZE is 1 the length field takes the value of PAYLOAD_LEN [15:0].

The following explains the meaning of PAYLOAD_BIT_ORDER in detail.

PAYLOAD_BIT_ORDER = 1 indicates that each byte of Payload and CRC is sent or Manchester/Whiten encoded in sequence from LSB to MSB during transmission. On the contrary, the MSB and LSB sequence of each byte of Payload and the CRC after decoding must be exchanged first during receiving, after then performing CRC decoding. If the configuration of RX and TX is the same, this processing is transparent to users. When it is need to use CMOSTEK product communicating with third-party products, users need to understand this process and have appropriate configurations.

PAYLOAD_BIT_ORDER = 0 indicates that there's no such processing.

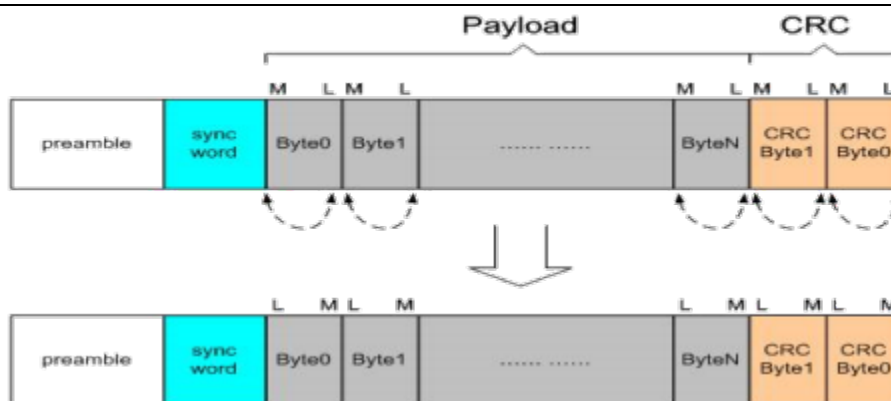


Figure 9. PAYLOAD_BIT_ORDER Operation

2.5 Address Configuration

The corresponding RFPDK GUI and parameters are shown as follow.

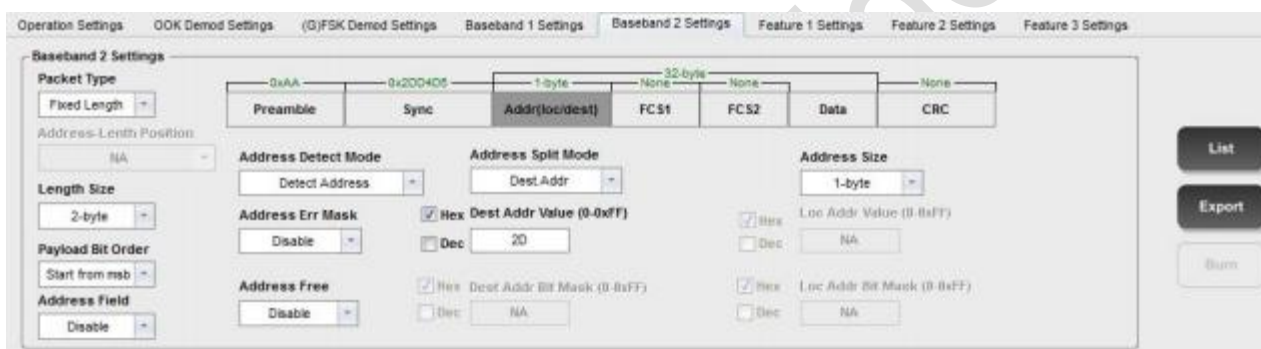


Figure 10. Address GUI on RFPDK

Table 13. Address Related Parameters

Register Bit RFPDK Parameter	Register Bit
Address Field Enable	ADDR_FIELD_EN
Address Detect Mode	ADDR_DET_MODE< 1:0>
Address Split Mode	ADDR_SPLIT_MODE
Address Size	ADDR_SIZE< 1:0>
Address Free	ADDR_FREE_EN
Local Address	LOCAL_ADDR < 15:0>
Destination Address	DEST_ADDR < 15:0>
Address Err Mask	ADDR_ERR_MASK
Destination Address Bit Mask	DEST_BITMASK< 15:0>
Source Address Bit Mask	SCR_BITMASK< 15:0>

Table 14. Registers in Configuration Area

Register Name	Bits	R/W	Bit Name	Function Description
CTL_REG_63 (0x3F)	3	RW	ADDR_FIELD_EN	Address field enable. 0 : no address field 1 : with address field
CTL_REG_64 (0x40)	6	RW	ADDR_SPLT_MODE	Spilt address mode configuration. 0 : only DEST ADDR field is applied and at this time, both {DEST_ADDR[15:0] and LOCAL_ADDR[15:0] } can be used as DEST ADDR 1 : DEST ADDR field + SRC ADDR field and at this time, DEST_ADDR[15:0] is applied to configure DEST ADDR, LOCAL_ADDR[15:0] is applied to configure SRC ADDR.
	5	RW	ADDR_FREE_EN	In RX mode, using the enabling bit to separate ADDRESS detection circuit, that is, keep detecting and validating address during the receiving. 0: disable 1: enable
	4	RW	ADDR_ERR_MASK	ADDRESS error detecting will output PKT_ERR interrupt and it can reset decoding circuit at the same time to make sure whether the bit is reset synchronously. 0: enable reset at the same time. 1: disable reset at the same time.
	3:2	RW	ADDR_SIZE< 1:0>	ADDRESS length. 0 : 1 byte 1 : 2 bytes 2 : 3 bytes 3 : 4 bytes Note: when ADDR_SPLIT_MODE is 1 , it means DEST ADDR field and SRC ADDR field takes up 1 or 2 bytes respectively; when ADDR_SPLIT_MODE is 0 , DEST ADDR field is optional from 1~4 bytes.
	1:0	RW	ADDR_DET_MODE< 1:0>	ADDRESS detection mode. When ADDR_SPLIT_MODE is 1 , 0 : In TX mode: sends the content of DEST_ADDR+LOCAL_ADDR. In RX mode, only the contents of the ADDR field are received. No address matching adjustment. 1 : In TX mode: sends the content of DEST_ADDR+LOCAL_ADDR; In RX mode, only content of the ADDR field are received, that is, matching LOCAL_ADDR in RX with DEST_ADDR in RX. ; while DEST_ADDR in RX with LOCAL_ADDR in

Register Name	Bits	R/W	Bit Name	Function Description
				TX. 2 : Sends the content of DEST_ADDR+LOCAL_ADDR in TX mode and recognizes the contents of the ADDR field and all zeros in RX mode, that is, in addition to matching configuration 1, all zero can also be matched correctly 3 : Sends the content of DEST_ ADDR+LOCAL_ADDR in TX mode and recognizes the contents of the ADDR field with all 0 and all 1 in RX mode, that is, in addition to matching configuration 1, ADDR all 0 or all 1 can also be matched correctly. When ADDR_SPLIT_MODE is 0, 0 : sends out content of {DEST ADDR, LOCAL_ADDR} in TX mode; only receive ADDR filed content in RX mode, no address matching adjustment. 1 : TX mode, send out content of {DEST ADDR, LOCAL_ADDR}; RX mode, only recognize content of ADDR field. 2 : TX mode, send out content of {DEST ADDR, LOCAL_ADDR}; RX mode, only recognize content of ADDR field and all 0. 3 : TX mode, send out content of {DEST ADDR, LOCAL_ADDR}; RX mode, recognize content of ADDR field, including all 0 and all 1.
CTL_REG_65 (0x41)	7:0	RW	LOCAL_ADDR <7:0>	16 bit local ADDRESS value When ADDR_SPLIT_MODE is 1 , it is only used for LOCAL ADDR field.
CTL_REG_66 (0x42)	7:0	RW	LOCAL_ADDR < 15:8>	When ADDR_SPLIT_MODE is 0 , it can be used as the lower 16 bit mask of DEST ADDR configuration.
CTL_REG_67 (0x43)	7:0	RW	DEST_ADDR <23:16>	16 bit destination ADDRESS value. When ADDR_SPLIT_MODE is 1 , it is only used for DEST ADDR field.
CTL_REG_68 (0x44)	7:0	RW	DEST_ADDR <31:24>	When ADDR_SPLIT_MODE is 0 , it can be used as the higher 16 bit mask of DEST ADDR configuration.
CTL_REG_69 (0x45)	7:0	RW	SCR_BITMASK<7:0>	16 bit source address bit mask. If the configuration bit mask is set to 1, the corresponding ADDR will not compare to each other.
CTL_REG_70 (0x46)	7:0	RW	SCR_BITMASK< 15:8>	When ADDR_SPLIT_MODE is 1 , it is only used for SCR ADDR field. When ADDR_SPLIT_MODE is 0 , it can be used as the lower 16 bit mask of DEST ADDR configuration.
CTL_REG_71 (0x47)	7:0	RW	DEST_BITMASK<23: 16>	16 bit destination bit mask. If the configuration bit mask is set to 1, the corresponding ADDR will not compare to

Register Name	Bits	R/W	Bit Name	Function Description
CTL_REG_72 (0x48)	7:0	RW	DEST_BITMASK<31:24>	each other. When ADDR_SPLIT_MODE is 1 , it is only used for DEST ADDR field. When ADDR_SPLIT_MODE is 0 , it can be used as the higher 16 bit mask of DEST ADDR configuration.

The configuration of ADDR has to be chosen according to ADDR_SPLIT_MODE , If ADDR_SPLIT_MODE is set to 0 , configurations are shown as follow:

	ADDR_VALUE{ DEST_ADDR<15:0>, LOCAL_ADDR<15:0> }			
ADDR_SIZE	<31:24>	<23: 16>	< 15:8>	<7:0>
0	√			
1	√	√		
2	√	√	√	
3	√	√	√	√

The tick in the table indicates the register required to be filled in. For example, if ADDR_SPLIT_MODE is set to 0 and ADDR_SIZE is set to 1 , that means it only has DEST ADDR field with length of 2 byte and value of 0x5678. Users will filled this value to registers of ADDR_VALUE<31:24> and ADDR_VALUE<23: 16>, with msb corresponding to 31st bit and lsb to 16th bit, that is , filled 0x56 into ADDR_VALUE<31:24> , and 0x78 into ADDR_VALUE<23: 16>.

If ADDR_SPLIT_MODE is set to 1 , configurations are shown as follow:

	DEST_ADDR<15:0> + LOCAL_ADDR<15:0>			
ADDR_SIZE	DEST_ADDR< 15:8>	DEST_ADDR<7:0>	LOCAL_ADDR< 15:8>	LOCAL_ADDR<7:0>
0	√		√	
1	√	√	√	√

The tick in the table indicates the register required to be filled in. For example, if ADDR_SPLIT_MODE is set to 1 and ADDR_SIZE is set to 1 , that means it only has DEST ADDR field and SRC ADDR field, with length of respectively 2 byte and DEST ADDR value of 0x5678, SRC ADDR value of 0x1234. Users will filled the DEST ADDR value to registers of DEST_ADDR< 15:8> and DEST_ADDR<7:0>, with msb corresponding to 15th bit and lsb to 0th bit, that is , filled 0x56 into DEST_ADDR< 15:8> , and 0x78 into DEST_ADDR<7:0> ; If value of SRC ADDR is filled into registers of both LOCAL_ADDR< 15:8> and LOCAL_ADDR<7:0>, msb will corresponds to 15th bit and lsb to 0th bit, that is, filled 0x12 into LOCAL_ADDR< 15:8> and 0x34 into LOCAL_ADDR<7:0>.

2.6 FCS1 Configuration

The corresponding RFPDK GUI and parameters are shown as follow.

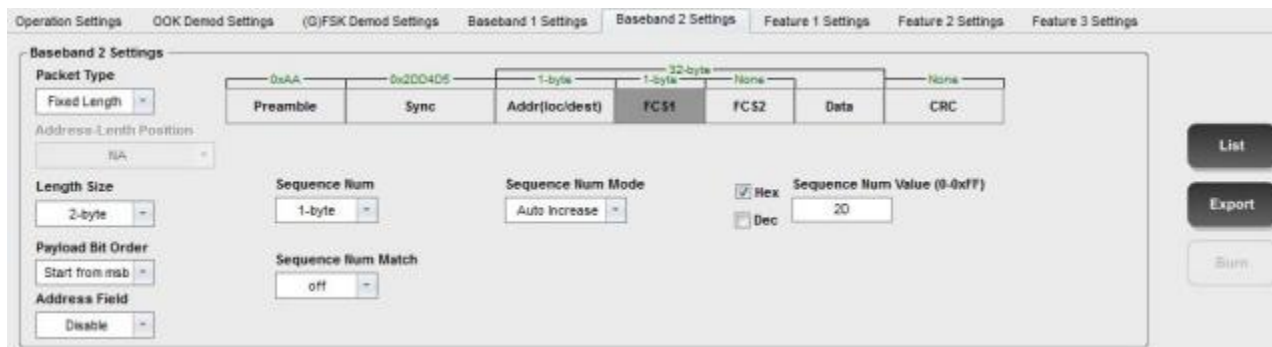


Figure 11. FCS1 GUI on RFPDK

Table 15. FCS1 Related Parameters

Register Bit RFPDK Parameter	Register Bit
When Sequence Num is None, SEQNUM_EN = 0; otherwise SEQNUM_EN = 1.	SEQNUM_EN
Sequence Num != None	SEQNUM_SIZE
Sequence Num Mode	SEQNUM_AUTO_INC
Sequence Num Value	SEQNUM_TX_IN< 15:0>
Sequence Num Match Detect	SEQNUM_MATCH_EN

Table 16. Registers in Configuration Area

Register Name	Bits	R/W	Bit Name	Function Description
CTL_REG_84 (0x54)	5	RW	SEQNUM_MATCH_EN	When Auto-ACK is enabled, the enabling of Master side can judge whether Slave side received packet successfully. 0: disable the judging 1: enable the judging
	4	RW	SEQNUM_SIZE	Sequence Num field length. 0: 1 byte. 1: 2 bytes.
	3	RW	SEQNUM_AUTO_INC	Whether Sequence Num is automatically increased. at TX side. 0: not increase 1: increase by 1 automatically.

	2	RW	SEQNUM_EN	0: disable Sequence Num field (FCS1 field) 1: enable Sequence Num field (FCS1 field)
CTL_REG_87 (0x57)	7:0	RW	SEQNUM_TX_IN<7:0>	The initial value of Sequence Num at TX side. When SEQNUM_SIZE is 0, SEQNUM_TX_IN< 15:8> is valid. When SEQNUM_SIZE is 1, SEQNUM_TX_IN< 15:0> is valid.
CTL_REG_88 (0x58)	7:0	RW	SEQNUM_TX_IN< 15:8>	

At TX side, Sequence Num is sent out as part of packet content; at RX side, it is filled into the mapped registers automatically, then users can read the register value directly.

The AUTO_ACK responding packet will be discussed below.

2.7 FCS2 Configuration

The corresponding RFPDK GUI and parameters are shown as follow.

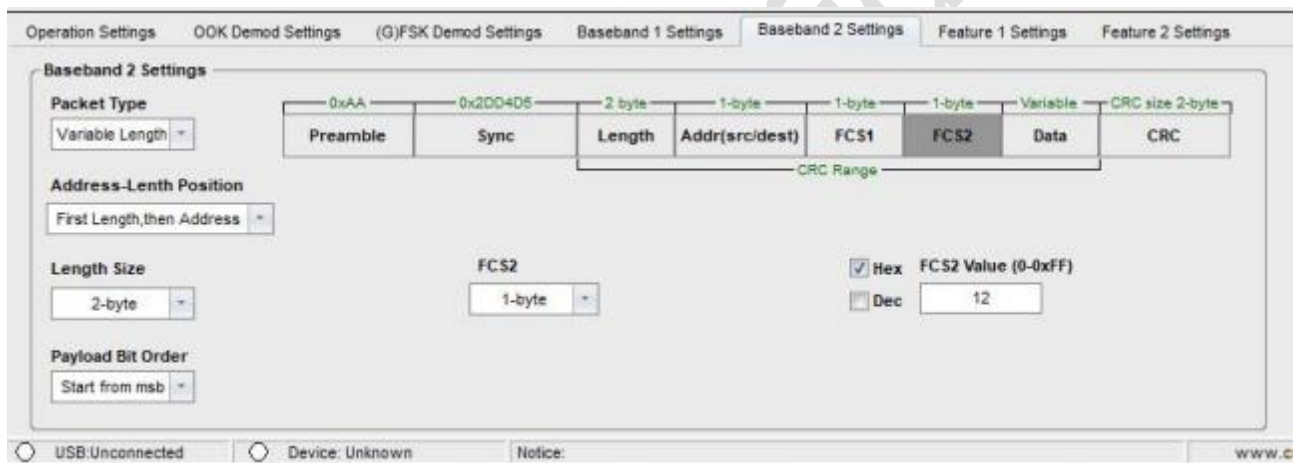


Figure 12. FCS2 GUI on RFPDK

Table 17. FCS2 Related Parameters

Register Bit RFPDK Parameter	Register Bit
FCS2	FCS2_EN
FCS2 Value	FCS2_TX_IN

Table 18. Registers in Configuration Area

Register Name	Bits	R/W	Bit Name	Function Description
CTL_REG_84 (0x54)	6	RW	FCS2_EN	0: disable FCS2 1: enable FCS2

CTL_REG_91 (0x5B)	7:0	RW	FCS2_TX_IN <7:0>	The transmitted FCS2 value at TX side. Here it defines FCS2_TX_IN[7] as the enabling bit (for Auto.Ack) for Master to ask Slave to send back ACK responding packet, that is, if Slave receives the bit 7 of FCS2 as 1 in a packet, it needs to send back ACK packet to the Master. FCS2_TX_IN[6:0] is reserved.
----------------------	-----	----	------------------	--

Here is a brief explanation of the auto answer, that is, the AUTO ACK function. The Slave is capable of automatically responding. When Slave receives a packet requiring ACK respond (that is receiving FCS2_TX_IN[7] as 1), then the chip will switch to TX mode and send back an ACK packet automatically with the packet format as shown in the figure below.



Figure 13. ACK Packet Format

The ACK packet structure contains Preamble and Sync ID. If the received packet structure contains Address and FCS1, the ACK packet will send out these two parts, where the Address field regards the SRC ADDR as destination address, and the RX local LOCAL_ADDR is sent back as the source address, and the FCS1 field returns the received Sequence Num directly.

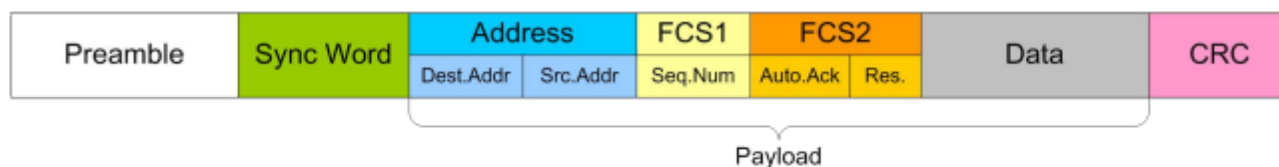
At Master side, if Slave is required to send back ACK packet, it needs to set field FCS2_TX_IN[7] as 1 and enters RX state automatically after completing current packet transmission and Slave sends back ACK packet. If SEQNUM_MATCH_EN is set as 1, it will compare whether the received Sequence Num and the local Sequence Num is consistent. If they are the same, it will generate SEQNUM_OK and users can read back this signal.

Table 19. Registers in Configuration Area

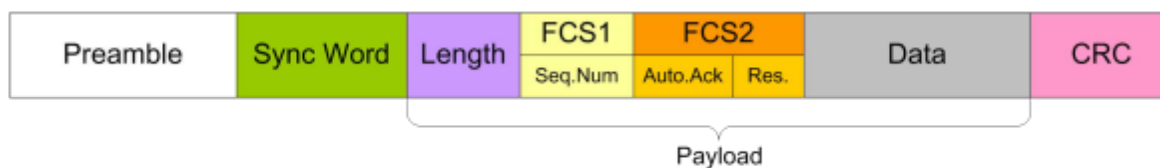
Register Name	Bits	R/W	Bit Name	Function Description
CTL_REG_96 (0x60)	2	RW	TX_ACK_EN	At TX side, enable the AUTO_ACK function of Master. 0: disable 1: enable
CTL_REG_97 (0x61)	2	RW	RX_ACK_EN	At RX side, enable the AUTO_ACK function of Slave. 0: disable 1: enable

TX_ACK_EN and RX_ACK_EN means that Master and Slave has the AUTO_ACK function. However, it is the bit 7 in FCS2 field depends on whether to transmit or receive the ACK packet.

Next section will introduce how to set PAYLOAD_LENGTH and ADDR_SIZE under different PKT_TYPE, that is, describe DATA length in Payload and implication of the Length Byte. Users can search for the needed part in the table below.

Fix length Packet Format:**Figure 14. Fixed Length Packet Format**

TX Mode	RX Mode
Payload structure: Address + FCS1 + FCS2 + Data, (Address / FCS1 / FCS2 can be absent while TX and RX configuration are identical) Example 1: PAYLOAD_LENGTH< 15:0> = 17 ADDR_DET_MODE< 1:0>!= 0 ADDR_SIZE< 1:0> = 2 ADDR_SPILT_MODE = 0 FCS1_EN = 1 SEQNUM_SIZE = 1 FCS2_EN = 1 Here Payload is comprised of 4 parts: Address + FCS1 + FCS2 + Data, with a total length of 17 + 1 = 18, The Address length is 2 (Dest.Addr) + 1 = 3, Sequence Num length is 1 + 1 = 2, and fixed length of FCS2 is 1, so Data length is 18 - 3 - 2 - 1 = 12 byte. If ADDR_SPILT_MODE = 1, it means the Address length is 0 (Dest.Addr) + 1 + 0 (Scr.Addr) + 1 = 2,that is,the length is 18 - 2 - 2 - 1 = 13 byte. Example 2: PAYLOAD_LENG< 15:0> = 17 ADDR_DET_MODE< 1:0> = 0 ADDR_SIZE< 1:0> = 2 ADDR_SPILT_MODE = 0 FCS1_EN = 0 SEQNUM_SIZE = 1 FCS2_EN = 0 Here , Address + FCS1 + FCS2 is absent, so that the Data length is 18.	

Variable length packet format:**Scenario 1: Address is absent****Figure 15. Variable Length Packet Format with Address Absent**

TX Mode	RX Mode
Payload structure: Length Byte + FCS1 + FCS2 + Data Payload length: In 1 + PAYLOAD_LENGTH< 15:0>, the content of PAYLOAD_LENGTH< 15:0> is equal to the content of Length Byte, standing for the length of the following Payload. If LENGTH_SIZE = 0, namely PAYLOAD_LENGTH< 15:8> is valid, with a maximum value of 255. The length of Length Byte field is 1 byte. If LENGTH_SIZE = 1, namely PAYLOAD_LENGTH< 15:0> is valid with a maximum value of 65535. The length of Length Byte field is 2 bytes. For example: PAYLOAD_LENGTH< 15:0> = 17 LENGTH_SIZE = 0 FCS1_EN = 1 SEQNUM_SIZE = 1 FCS2_EN = 1 Here, Payload is comprised of 4 parts, Length + (FCS1 + FCS2 + Data), with a total Payload length of 17 + 1 = 18. The length is 1, Sequence Num length is 1 + 1 = 2. The fixed FCS2 length is 1, therefore, the Data length is 17 + 1 - 2 - 1 = 15 byte. If FCS1 + FCS2 is absent, then the Data length is 17 + 1 = 18 byte. If LENGTH_SIZE = 1, it means the length is 2, so that the total length of Data is 17 + 1 + 2 = 20. If FCS1 + FCS2 is absent, Data length is 17 + 1 = 18 byte.	Payload structure: Length Byte + FCS1 + FCS2 + Data The implication of Length Byte: It shows Payload length after length field, which is corresponds to TX configuration.

Scenario 2: it contains Address field, and NODE_LENGTH_POS_SEL = 0 (Address is before Length Byte)



Figure 16. Variable Length Format Containing Address (Address is before Length Byte)

TX Mode	RX Mode
Payload structure: Address + Length Byte + FCS1 + FCS2 + Data Payload length: In (ADDR_SIZE< 1:0> + 1) + 1 + Length Byte size + PAYLOAD_LENGTH< 15:0>, the Address length is decided by ADDR_SPILT_MODE + ADDR_SIZE< 1:0> (please refer to the information mentioned above, the content of PAYLOAD_LENGTH< 15:0> is equal to content of Length Byte , which means the Payload length followed after the Length field. If LENGTH_SIZE = 0, it means that PAYLOAD_LENGTH< 15:8> is effective with a maximum value of 255 and a Length Byte of 1. If LENGTH_SIZE = 1, it means that PAYLOAD_LENGTH< 15:0> is effective with a maximum value of 65535 and a Length Byte of 2. For example: PAYLOAD_LENGTH< 15:0> = 17 ADDR_DET_MODE< 1:0> != 0 ADDR_SIZE< 1:0> = 2 ADDR_SPILT_MODE = 0 LENGTH_SIZE = 0 FCS1_EN = 1 SEQNUM_SIZE = 1 FCS2_EN = 1 Here , Payload is comprised of 5 parts, that is, Address + Length +	Payload structure: Address + Length Byte + FCS1 + FCS2 + Data The implication of Length Byte: It shows Payload length after length field, which is corresponds to TX configuration.

TX Mode	RX Mode
(FCS1 + FCS2 + Data). The Address length is 2 (Dest.Addr) + 1 = 3 , Length is 1 , Sequence Num length is 1 + 1 = 2 , while fixed length of FCS2 is 1. Then the total length of Payload is 3 + 1 + (17 + 1) = 22 and Data length is 17 + 1 - 2 - 1 = 15 byte. If FCS1 + FCS2 is absent, the Data length is 17 + 1 = 18 byte. If LENGTH_SIZE = 1, it means that Length is 2 , therefore the total length of Payload is 3 + 2 + (17 + 1) = 23. If Data length keep unchanged, it will be 17 + 1 - 2 - 1 = 15 byte. If FCS1 + FCS2 is absent, then the Data length is 17 + 1 = 18 byte.	

Scenario 3: it contains Address and NODE_LENGTH_POS_SEL = 1 (Address is after Length Byte)

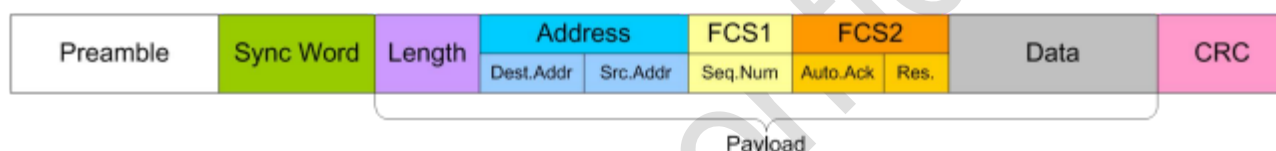


Figure 17. Variable Length Format Containing Address (Address is after Length Byte)

TX Mode	RX Mode
Payload structure: Length Byte + Address+ FCS1 + FCS2 + Data Payload length: In 1 + PAYLOAD_LENGTH< 15:0>, the content of PAYLOAD_LENGTH< 15:0> is equal to the content of Length Byte, that is the length of the following Payload. If LENGTH_SIZE = 0, namely PAYLOAD_LENGTH< 15:8> is valid with a maximum value of 255 and Length Byte of 1. If LENGTH_SIZE = 1, namely PAYLOAD_LENGTH< 15:0> is valid with a maximum value of 65535 and Length Byte of 2. For example: PAYLOAD_LENGTH< 15:0> = 17 ADDR_DET_MODE< 1:0> != 0	Payload structure: Length Byte + Address + FCS1 + FCS2 + Data The implication of Length Byte: It shows Payload length after length field, which is corresponds to TX configuration.

TX Mode	RX Mode
ADDR_SIZE< 1:0> = 2 ADDR_SPILT_MODE = 0 LENGTH_SIZE = 0 FCS1_EN = 1 SEQNUM_SIZE = 1 FCS2_EN = 1 Here, Payload is comprised of 5 parts, Length + (Address + FCS1 + FCS2 + Data) , with a total Payload length of 17 + 1 + 1 = 19. The length is 1, Address length is 2 (Dest.Addr) + 1 = 3 , Sequence Num length is 1 + 1 = 2, fixed FCS2 length is 1, therefore, the Data length is 17 + 1 - 3 - 2 - 1 = 12 byte. If FCS1 + FCS2 is absent, then the Data length is 17 + 1 - 3 = 15 byte. If LENGTH_SIZE = 1, it means the length is 2, so that the total length of Data is 17 + 1 + 2 = 20. Data length is 17 + 1 - 3 - 2 - 1 = 12 byte. If FCS1 + FCS2 is absent, then the Data length is 17 + 1 - 3 = 15 byte.	

2.8 CRC Configuration

The corresponding RFPDK GUI and parameters are shown as follow.

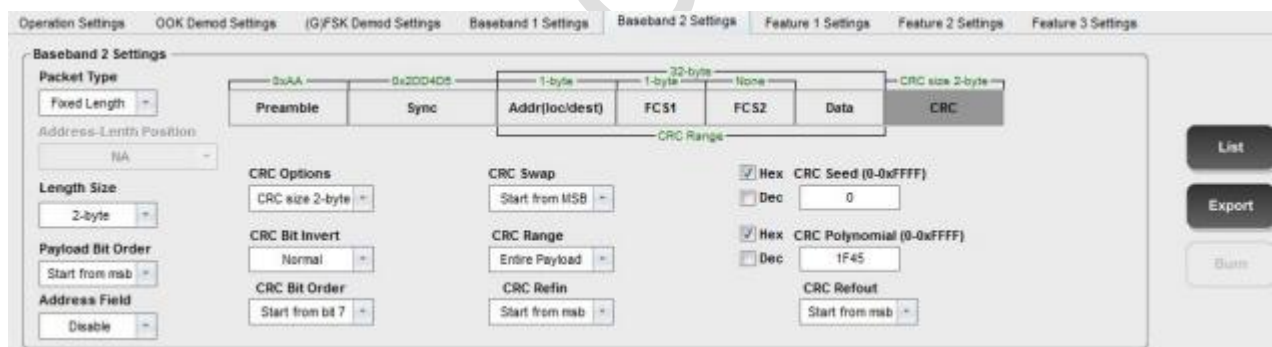


Figure 18. CRC GUI on RFPDK

Table 20. CRC Related Parameters

Register Bit RFPDK Parameter	Register Bit
Crc Options != None	CRC_SIZE< 1:0>
When Crc Options is Non, CRC_EN = 0; otherwise CRC_EN =1	CRC_EN
Crc Seed	CRC_SEED<31:0>
Crc Polynomial	CRC_POLY<31:0>
Crc Range	CRC_RANGE
Crc Byte Swap	CRC_BYTE_SWAP

Crc Bit Invert	CRC_BIT_INV
Crc Bit Order of Byte	CRC_ORDER
Crc Refin	CRC_REFIN
Crc Refout	CRC_REFOUT

Table 21. Registers in Configuration Area

Register Name	Bits	R/W	Bit Name	Function Description
CTL_REG_73 (0x49)	7:6	RW	CRC_SIZE< 1:0>	The byte number of CRC field check code. 0: 1 byte 1: 2 bytes 2: 3 bytes 3: 4 bytes
	5	RW	CRC_BYTE_SWAP	CRC transmitting and receiving sequence. 0: transmit & receive starting from higher byte. 1: transmit & receive starting from lower byte.
	4	RW	CRC_BIT_INV	Whether have reverse of CRC code. 0: not have reverse of CRC code. 1: have reverse of CRC code bit by bit.
	3	RW	CRC_RANGE	CRC calculation range. 0: entire payload. 1: only data field.
	2	RW	CRC_REFIN	The sequence of bit reverse of input byte during CRC calculation. 0: from MSB to LSB. 0: from LSB to MSB.
	1	RW	CRC_ORDER	Big-Endian and Little-Endian sequence inside CRC bytes. 0: receive and send from higher bit. 1: receive and send from lower bit.
	0	RW	CRC_EN	CRC enabling. 0: disable. 1: enable.
CTL_REG_74 (0x4A)	7:0	RW	CRC_SEED<7:0>	The initial value of CRC polynomial.
CTL_REG_75 (0x4B)	7:0	RW	CRC_SEED< 15:8>	
CTL_REG_76 (0x4C)	7:0	RW	CRC_SEED<23: 16>	
CTL_REG_77 (0x4D)	7:0	RW	CRC_SEED<31:24>	
CTL_REG_78 (0x4E)	7:0	RW	CRC_POLY<7:0>	CRC calculation polynomial, supporting any polynomial configuration within 32 bits.
CTL_REG_79 (0x4F)	7:0	RW	CRC_POLY< 15:8>	
CTL_REG_80 (0x50)	7:0	RW	CRC_POLY<23: 16>	

CTL_REG_81 (0x51)	7:0	RW	CRC_POLY<31:24>	
CTL_REG_82 (0x52)	7	RW	CRC_REFOUT	During CRC calculation, the sequence of all output bits reverse. 0: from MSB to LSB. 1: from LSB to MSB.

The following describes how CRC configurations work in detail

CRC_RANGE

This feature is used to specifies the codec CRC checking object, which could be an entire Payload or just a Data part.

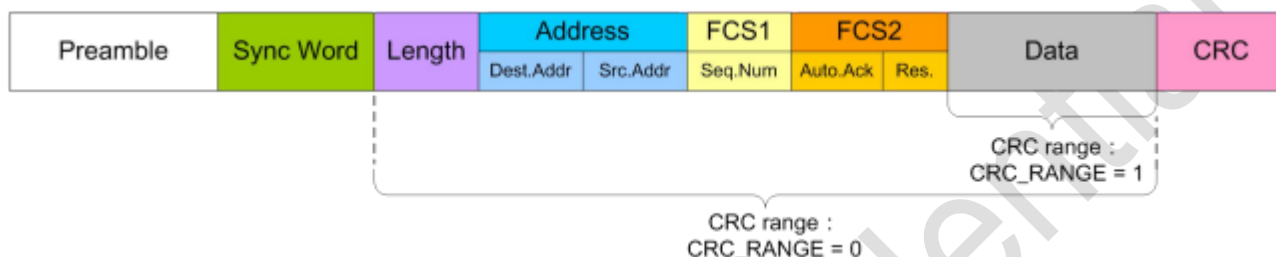


Figure 19. CRC Coding/Decoding Range

CRC_BIT_INV

This function is to have logic reverse of every bit in CRC, that is, change 0 to 1 and 1 to 0.

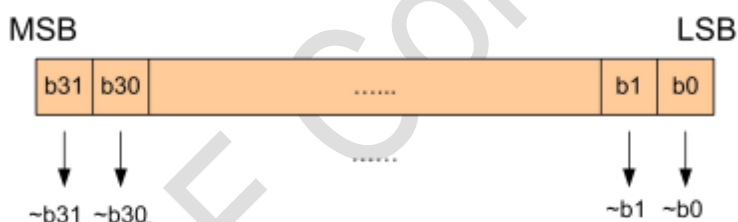


Figure 20. CRC_BIT_INV

CRC_BYTE_SWAP

This function is to reverse the position of the 4 Byte without changing Bit sequence of each Byte.

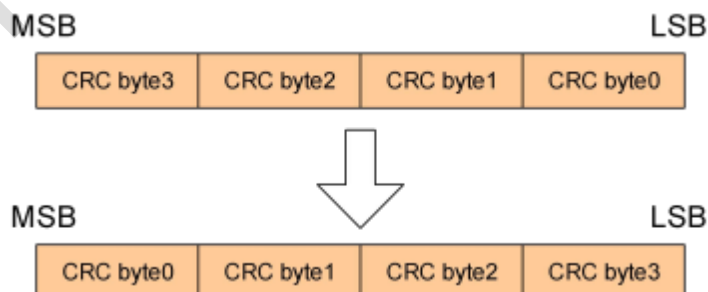


Figure 21. CRC_BYTE_SWAP

CRC_BIT_ORDER

This function reverses the bit sequence from higher to lower inside each byte among the entire CRC part.

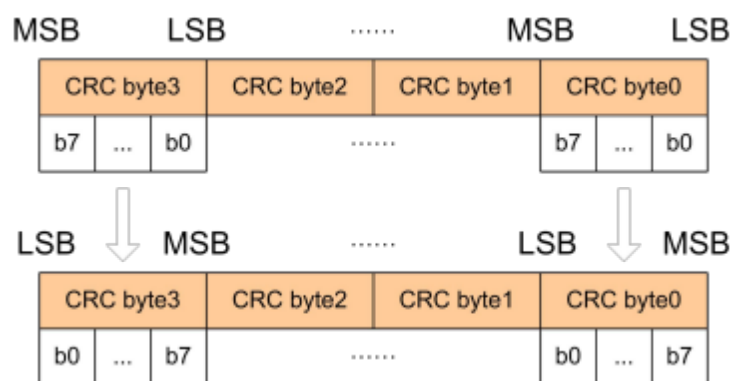


Figure 22. CRC_BIT_ORDER

2.9 CODEC Configuration

The corresponding RFPDK GUI and parameters are shown as follow

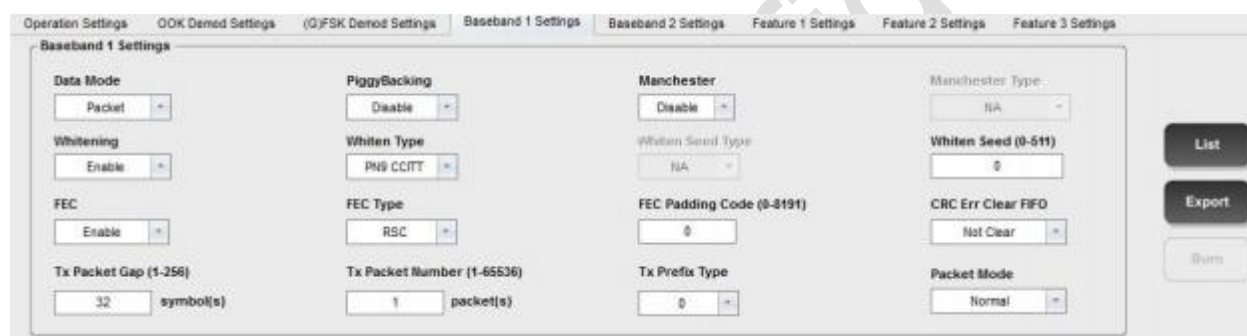


Figure 23. CODEC on RFPDK GUI

Table 22. CODEC Related Parameters

Register Bit RFPDK Parameter	Register Bit
Whitening	WHITEN_EN
Whiten Type	WHITEN_TYPE< 1:0>
Whiten Seed Type	WHITEN_SEED_TYPE
Whiten Seed	WHITEN_SEED<8:0>
Manchester	MANCH_EN
Manchester Type	MANCH_TYPE
FEC	FEC_EN
FEC Type	FEC_RSC_NRNSC_SEL
NRNSC Type	FEC_TICC
FEC Padding Code	FEC_PAD_CODE
Paggy Backing	PAGGYBACKING_EN

Table 23. Registers in Configuration Area

Register Name	Bits	R/W	Bit Name	Function Description
CTL_REG_82 (0x52)	6	RW	WHITEN_SEED<8>	The eighth bit of the whitening CODEC polynomial seed. When the whitening CODEC mode is PN9, the seed takes entirely 9 bits. When it is PN7, it takes the lower 7 bits.
	5	RW	WHITEN_SEED_TYPE	When the whitening CODEC mode is PN7: 0: calculate PN7 seed according to the calculation method in A7139. 1: the PN7 seed is the value defined in <code>whiten_seed</code> .
	4:3	RW	WHITEN_TYPE< 1:0>	The whitening encoding/decoding mode. 0: PN9 CCITT encoding/decoding. 1: PN9 IBM encoding/decoding. 2: PN7 encoding/decoding. 3: invalid
	2	RW	WHITEN_EN	The enabling of whitening encoding/decoding. 0: disable whitening encoding/decoding. 1: enable whitening encoding/decoding,
	1	RW	MANCH_TYPE	Manchester encoding/decoding mode. 0: 01 stands for 1; 10 stands for 0. 1: 10 stands for 1; 01 stands for 0.
	0	RW	MANCH_EN	The enabling of Manchester encoding/decoding. 0: disable. 1: enable.
CTL_REG_83 (0x53)	7:0	RW	WHITEN_SEED<7:0>	The bits 7:0 of the whitening encoding/decoding polynomial seed.
CTL_REG_93 (0x5D)	7	RW	FEC_TICC	Polynomial selection of NRNSC: 0 : shown as the polynomial structure in Figure 24, u_i takes the reverse output 1: shown as the polynomial structure in Figure 25, u_i don't takes the reverse output
	6:2	RW	FEC_PAD_CODE< 12: 8>	12:8 higher bit of the FEC Padding code
	1	RW	FEC_RSC_NRNSC_SEL	FEC polynomial selection: 0 : RSC 1 : NRNSC
	0	RW	FEC_EN	FEC CODEC enabling: 0: no FEC CODEC 1: with FEC CODEC
CTL_REG_94 (0x5E)	7:0	RW	FEC_PAD_CODE<7:0>	7:0 lower bit of the FEC Padding code

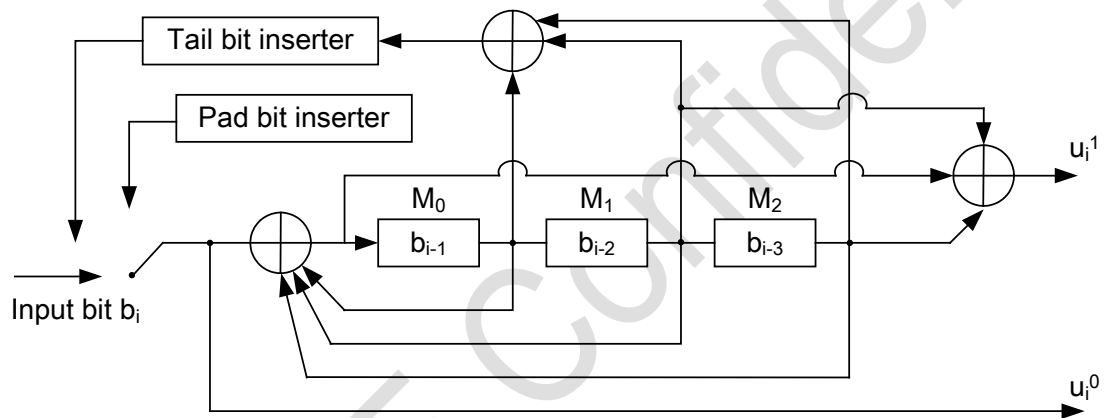
CTL_REG_63 (0x3F)	4	RW	PAGGYBACKING_EN	Whether the ACK packet has payload: 0: no payload 1: with payload
----------------------	---	----	-----------------	---

The application of FEC CODEC is described below.

The FEC calculation range is (Length + Address + FCS1 + FCS2 + Data + CRC) + Tail + Padding. Tail + Padding is inserted when FEC is making calculation. Tail makes sure the status of FEC is 0 when FEC complete coding. Padding guarantee the integrity of mixed operation; If the total length of Length + Address + FCS1 + FCS2 + Data + CRC is an odd number, then take FEC_PAD_CODE[4:0] ; if an even number, then take FEC_PAD_CODE[12:0].

FEC can be select from two types of poly-nomial.:

RSC:



$$u_i^0 = b_i$$

$$u_i^1 = b_i \oplus (b_{i-1} \oplus b_{i-2} \oplus b_{i-3}) \oplus b_{i-2} \oplus b_{i-3}$$

Figure 24. RSC Encoding

NRNSC:

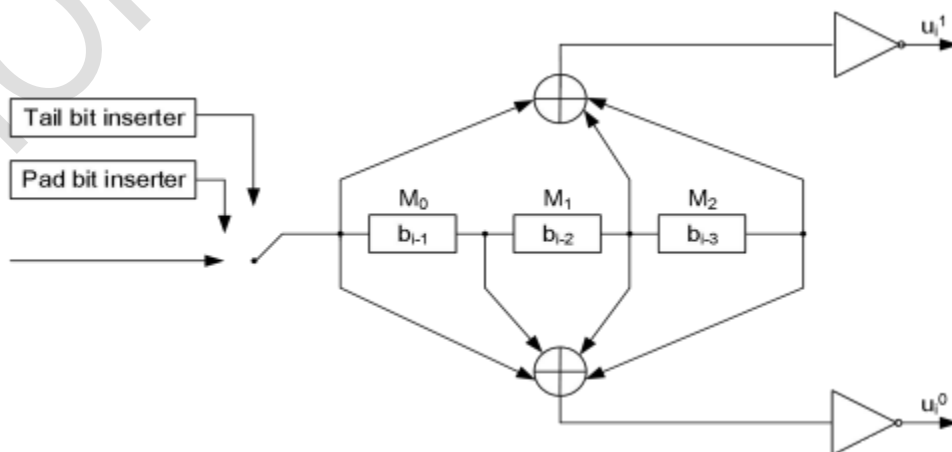


Figure 25. NRNSC encoding

$$u_i^1 = \sim(b_i \oplus b_{i-2} \oplus b_{i-3})$$

$$u_i^0 = \sim(b_i \oplus b_{i-1} \oplus b_{i-2} \oplus b_{i-3})$$

Both RSC and NRNSC will have mixed processing after encoding. Although whitening is optional, Manchester encoding and FEC as well as whitening encoding cannot be applied at the same time. When FEC encoding is applied, the receiving side will have decoding through Viterbi decoding.

2.10 Wi-sun Data Packet Configuration

The Wi-sun packet structure is shown as follow, please refer to the 802.15.4g protocol for more information.

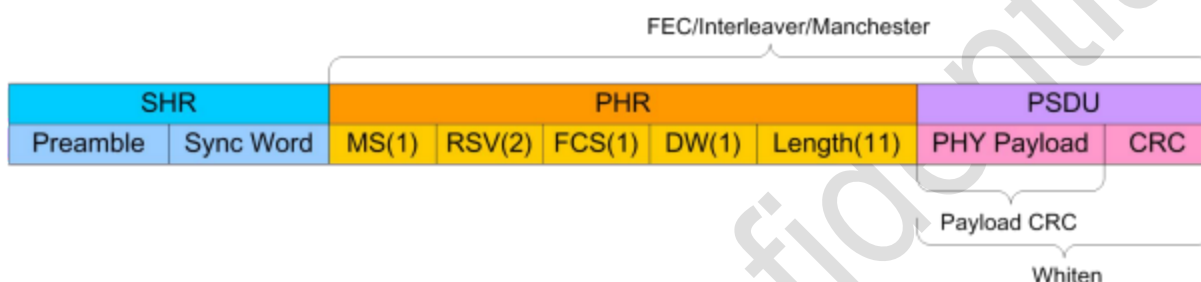


Figure 26. Wi-sun Packet Structure

The corresponding RFPDK GUI and parameters are shown as follow:

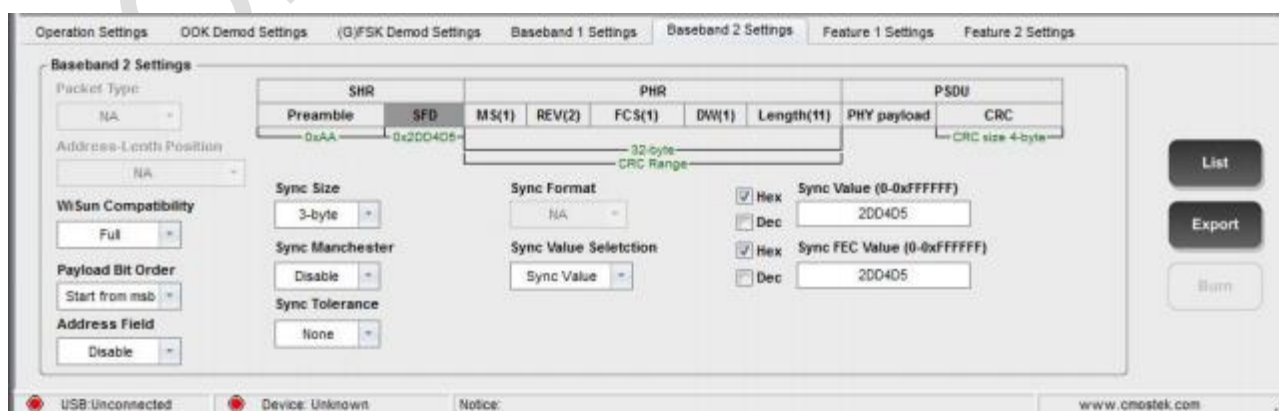


Figure 27. Data Packet on RFPDK

Table 24. Related Parameters of Data Packet

Register Bit RFPDK Parameter	Register Bit
Packet Mode	LENGTH_MODE
WiSun Compatibility	WISUN_ALLIN
WiSun Whiten Polynomial Select	WHITEN_WISUN
WiSun MS	WISUN_MS
WiSun REV	WISUN_RSV
WiSun FCS	WISUN_FCS
WiSun DW	WISUN_DW
WiSun Length	WISUN_LENGTH< 10:0>

Table 25. Registers in Configuration Area

Register Name	Bits	R/W	Bit Name	Function Description
CTL_REG_111 (0x6F)	7	RW	LENGTH_MODE	Length field structure selection 0: CMT2310A normal packet structure, value of Length field equals to the packet length 1: shown in Figure 26, higher 5 bit of Length field corresponds to CTL_REG_111[4:0] and lower 1 bit corresponds to the PSDU field length.
	6	RW	WISUN_ALLIN	Configuration source of both FCS field and DW field. 0: FCS field and DW is decided by the system configuration, while WISUN_FCS and WISUN_DW are invalid. 1: Both FCS field and DW field are decided by WISUN_FCS and WISUN_DW, which is completely compatible with Wi-sun protocol.
	5	RW	WHITEN_WISUN	Whitening polynomial selection 0 : CMT2310A packet supports for 3 kinds of whitening polynomial 1 : whitening polynomial supported by Wi-sun protocol
	4	RW	WISUN_MS	Default as 0
	3:2	RW	WISUN_RSV	Reserved
	1	RW	WISUN_FCS	CRC length of PSDU field 0: 4Bytes 1: 2Bytes
	0	RW	WISUN_DW	PSDU field whitening enabling 0 : whitening disabled 1 : whitening enabled

CTL_REG_61 (0x3D)	7:0	RW	PAYLOAD_LENGTH<7:0>	<7:0> bit of the 11-bit PHY Payload length
CTL_REG_62 (0x3E)	2:0	RW	PAYLOAD_LENGTH< 10:8>	< 10:8> bit of the 11-bit PHY Payload length

CMT2310A is basically fully compatible with the Wi-SUN MR- FSK of the 802. 15.4g protocol and can implement Wi-SUN packets according to RFPDK easy configuration. There are two points to be noted, firstly, when the PHY payload length is lower than 4Byte and WISUN_FCS is set to 0, it means the CRC length is 4 bytes. While 802. 15.4g protocol requires 0 calculation, and CMT2310A does not support 0; Secondly, FEC and mixed operation has to enable simultaneously.

2.11 Special Configuration for TX Data Packets

The corresponding RFPDK GUI and parameters are shown as follow.

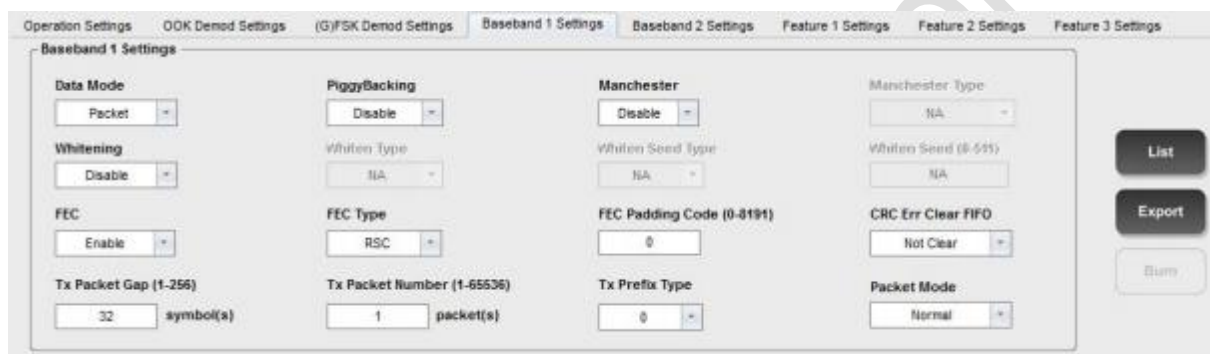


Figure 28. TX Data Packet on RFPDK GUI

Table 26. TX Data Packet Related Parameters

Register Bit	RFPDK Parameter	Register Bit
Tx Packet Number		TX_PKT_NUM< 15:0>
Tx Packet Gap		TX_PKT_GAP<7:0>

Table 27. Registers in Configuration Area

Register Name	Bits	R/W	Bit Name	Function Description
CTL_REG_85 (0x55)	7:0	RW	TX_PKT_NUM<7:0>	The number of packets transmitted repeatedly each time in TX mode: 0-65535 stands for transmitting 1-65536 packets.
CTL_REG_86 (0x56)	7:0	RW	TX_PKT_NUM< 15:8>	
CTL_REG_86 (0x56)	7:0	RW	TX_PKT_GAP<7:0>	Interval between packets when repeated transmitting under TX mode: 0-255 stands for 1-256 Symbol of interval between packets while transmitting.

2.12 Direct Transmission Mode

As discussed above, in TX mode, when DATA_MODE is configured as Direct, the transmitted data is from GPIO directly. The registers related to Direct TX mode are listed in the table below.

Table 28. Register Configured under Direct TX Mode

Register Name	Bits	R/W	Bit Name	Function Description
CTL_REG_40 (Page0, 0x28)	1:0	RW	DATA_MODE< 1:0>	Select RX and TX data mode. 0: Direct mode (default) 1: NA 2: Packet mode 3: NA
TX_DR_REG_02 (Page1, 0x17)	7	RW	TX_DATA_MODE	Select TX data source. 0: TX data is from GPIO directly. 1: TX data is fetched from TX FIFO.

Table 29. Related Control Register under Direct TX Mode

Register Name	Bits	R/W	Bit Name	Function Description
CTL_REG_4 (0x04)	6	RW	TX_EN	0: disable TX data from GPIO input 1: enable TX data from GPIO input
CTL_REG_5 (0x05)	7:6	RW	TX_SEL< 1:0>	Select GPIO input for data transmitting: 00: GPIO3 01: GPIO4 10: NIRQ 11: NA
	5:3	RW	GPIO3_SEL<2:0>	Select the function of GPIO3: 000: INT2 001: INT1 010: DCLK 011: DOUT Other option: NA
CTL_REG_6 (0x06)	2:0	RW	GPIO4_SEL<2:0>	Select the function of GPIO4: 000: DOUT 001: INT1 010: INT2 011: DCLK Other option: NA

CTL_REG_7 (0x07)	2:0	RW	NIRQ_SEL<2:0>	Select the function of NIRQ: 000: INT1 001: INT2 010: DCLK 011: DOUT 100: TCXO Other option: NA
CTL_REG_22 (0x16)	4	RW	TX_DATA_INV	0: do not reverse TX data input from GPIO 1: reverse TX data input from GPIO

Users has to set both DATA_MODE and TX_DATA_MODE as 0.

Users has to set the registers ready before transmitting. After sending out the GO_TX command, the external MCU sends the data from the corresponding GPIO. It should be noted that, in Direct mode, the data transmission rate is controlled by MCU and it needs to be as close to the configured chip data rate as possible.

Besides, users can use DCLK to assist data synchronization. It is suggested that MCU sends out data on the falling edge of DCLK. When data transmission completes, it can send the GO_* command to switch mode. In general, when the chip receives the GO_* command, it takes 2-3 symbol time to conduct the operation of PA RAMP DOWN. After then it can switch to the target state.

3. GPIO and Interrupt

CMT2310A supports 6 GPIOs, with which can be configured to input or output. It supports 2 interrupt ports, which can be configured to different GPIO output. More information will be described in the following section.

3.1 GPIO Configuration

The GPIO configuration related registers are described in the table below

Table 30. GPIO Related Registers

Register Name	Bits	R/W	Bit Name	Function Description
	5:3	RW	GPIO1_SEL<2:0>	Select the GPIO1 function: 000: DCLK 001: INT1 010: INT2 011: DOUT Other option: NA

CTL_REG_4 (0x04)	2:0	RW	GPIO0_SEL<2:0>	Select the GPIO 0 function: 000: DOUT 001: INT1 010: INT2 011: DCLK 111: INT3 Other option: NA
CTL_REG_5 (0x05)	5:3	RW	GPIO3_SEL<2:0>	Select the GPIO 3 function: 000: INT2 001: INT1 010: DCLK 011: DOUT Other option: NA
	2:0	RW	GPIO2_SEL<2:0>	Select the GPIO 2 function: 000: INT1 001: INT2 010: DCLK 011: DOUT 111: INT3 Other option: NA
CTL_REG_6 (0x06)	6	RW	DIG_CLKOUT_EN	Configure GPIO 4 as digital clock output: 0 : disable 1 : output This function takes priority to the other configuration of GPIO 4.
	5:3	RW	GPIO5_SEL<2:0>	Select the GPIO 5 function: 000: RSTn 001: INT1 010: INT2 011: DOUT 100: DCLK Other option: NA
	2:0	RW	GPIO4_SEL<2:0>	Select the GPIO 4 function: 000: DOUT 001: INT1 010: INT2 011: DCLK Other option: NA
	5	RW	LFXO_PAD_EN	Set GPIO 2 and GPIO3 as pins of LFXO: 0 : disable 1 : enable This function takes priority to the other configurations of GPIO 2 and GPIO 3.

CTL_REG_7 (0x07)	2:0	RW	NIRQ_SEL<2:0>	Select the NIRQ function: 000: INT1 001: INT2 010: DCLK 011: DOUT 100: TCXO Other option: NA
---------------------	-----	----	---------------	--

It should be noted that in the table above, DCLK is the synchronization clock for MCU transmitting or receiving data in Direct mode. When it enters RX or TX state, it will switch to synchronization RX or TX automatically.

3.2 Interrupt Configuration and Mapping

The CMT2310A supports two interrupt ports, INT1 and INT2, which can be distributed to different GPIO. The interrupt related registers are listed below.

Table 31. Interrupt Related Register

Register Name	Bits	R/W	Bit Name	Function Description
CTL_REG_14 (0x0E)	7	R	RX_FIFO_FULL_EN	Interrupt enable when RX FIFO is full 0 : disable 1 : enable
	6	R	RX_FIFO_NMTY_EN	Interrupt enable when RX FIFO in not full 0 : disable 1 : enable
	5	R	RX_FIFO_TH_EN	Interrupt enable when RX FIFO unread content exceed FIFO TH. 0 : disable 1 : enable
	4	R	RESV	Reserve bit with value of 0.
	3	R	RX_FIFO_OVF_EN	Interrupt enable when RX FIFO overflow. 0 : disable 1 : enable
	2	R	TX_FIFO_FULL_EN	Interrupt enable when TX FIFO in not full 0 : disable 1 : enable
	1	R	TX_FIFO_NMTY_EN	Interrupt enable when TX FIFO in not full 0 : disable 1 : enable
	0	R	TX_FIFO_TH_EN	Interrupt enable when TX FIFO unread content exceed FIFO TH. 0 : disable 1 : enable
CTL_REG_16 (0x10)	5:0	RW	INT1_SEL<5:0>	INT 1 mapping selection. More info please refer to the Interrupt Mapping Table.

CTL_REG_17 (0x11)	7	RW	INT1_POLAR	Polarity of interrupt 1. 0 : High effective 1 : Low effective
	6	RW	INT2_POLAR	Polarity of interrupt 2: 0 : High effective 1 : Low effective
	5:0	RW	INT2_SEL<5:0>	INT 2 mapping selection. More info please refer to the Interrupt Mapping Table.
CTL_REG_18 (0x12)	7	RW	SLEEP_TMO_EN	0 : Disable SLEEP_TMO interrupt 1 : Enable SLEEP_TMO interrupt
	6	RW	RX_TMO_EN	0 : Disable RX_TMO interrupt 1 : Enable RX_TMO interrupt
	5	RW	TX_DONE_EN	0 : Disable TX_DONE interrupt 1 : Enable TX_DONE interrupt
	4	RW	PREAM_PASS_EN	0 : Disable PREAM_PASS interrupt 1 : Enable PREAM_PASS interrupt
	3	RW	SYNC_PASS_EN	0 : Disable SYNC_PASS interrupt 1 : Enable SYNC_PASS interrupt
	2	RW	ADDR_PASS_EN	0 : Disable ADDR_PASS interrupt 1 : Enable ADDR_PASS interrupt
	1	RW	CRC_PASS_EN	0 : Disable CRC_PASS interrupt 1 : Enable CRC_PASS interrupt
	0	RW	PKT_DONE_EN	0 : Disable PKT_DONE interrupt 1 : Enable PKT_DONE interrupt
CTL_REG_19 (0x13)	7	RW	INT3_POLAR	Polarity of interrupt 3: 0 : High effective 1 : Low effective
CTL_REG_21 (0x15)	6	RW	RSSI_PJD_VALID_EN	0 : Disable RSSI_PJD_VALID interrupt 1 : Enable RSSI_PJD_VALID interrupt
	5	RW	OP_CMD_FAILED_EN	0 : Disable API_CMD_FAILED interrupt 1 : Enable API_CMD_FAILED interrupt
	4	RW	RSSI_COLL_EN	0 : Disable RSSI_COLL interrupt 1 : Enable RSSI_COLL interrupt
	3	RW	PKT_ERR_EN	0 : Disable PKT_ERR interrupt 1 : Enable PKT_ERR interrupt
	2	RW	LBD_STATUS_EN	0 : Disable LBD_STATUS interrupt 1 : Enable LBD_STATUS interrupt
	1	RW	LBD_STOP_EN	0 : Disable LBD_STOP interrupt 1 : Enable LBD_STOP interrupt
	0	RW	LD_STOP_EN	0 : Disable LD_STOP interrupt 1 : Enable LD_STOP interrupt
CTL_REG_22 (0x16)	0	RW	ANT_LOCK_EN	0 : Disable ANT_LOCK interrupt 1 : Enable ANT_LOCK interrupt
	7	RW	API_DONE_EN	0 : Disable API_DONE interrupt 1 : Enable API_DONE interrupt

CTL_REG_23 (0x17)	6	RW	CCA_STATUS_EN	0 : Disable CCA_STATUS interrupt 1 : Enable CCA_STATUS interrupt
	5	RW	CSMA_DONE_EN	0 : Disable CSMA_DONE interrupt 1 : Enable CSMA_DONE interrupt
	4	RW	TX_DC_DONE_EN	0 : Disable TX_DC_DONE interrupt 1 : Enable TX_DC_DONE interrupt
	3	RW	ACK_RECV_FAILED_EN	0 : Disable ACK_RECV_FAILED interrupt 1 : Enable ACK_RECV_FAILED interrupt
	2	RW	TX_RESEND_DONE_EN	0 : Disable TX_RESEND_DONE interrupt 1 : Enable TX_RESEND_DONE interrupt
	1	RW	NACK_RECV_EN	0 : Disable NACK_RECV interrupt 1 : Enable NACK_RECV interrupt
	0	RW	SEQ_MATCH_EN	0 : Enable SEQ_MATCH interrupt 1 : Enable SEQ_MATCH interrupt
CTL_REG_24 (0x18)	5	R	SLEEP_TMO_FLG	SLEEP_TMO interrupt flag
	4	R	RX_TMO_FLG	RX_TMO interrupt flag
	3	R	TX_DONE_FLG	TX_DONE interrupt flag
	2	W	SLEEP_TMO_CLR	SLEEP_TMO interrupt clearing 0: no action. 1: clearing.
	1	W	RX_TMO_CLR	RX_TMO interrupt clearing 0: no action. 1: clearing.
	0	W	TX_DONE_CLR	TX_DONE interrupt clearing 0: no action. 1: clearing.
CTL_REG_25 (0x19)	4	W	PREAM_PASS_CLR	PREAM_PASS interrupt clearing 0: no action. 1: clearing.
	3	W	SYNC_PASS_CLR	SYNC_PASS interrupt clearing 0: no action. 1: clearing.
	2	W	ADDR_PASS_CLR	ADDR_PASS interrupt clearing 0: no action. 1: clearing.
	1	W	CRC_PASS_CLR	CRC_PASS interrupt clearing 0: no action. 1: clearing.
	0	W	PKT_DONE_CLR	PKT_DONE interrupt clearing 0: no action. 1: clearing.
CTL_REG_26 (0x1A)	5	R	SYNC1_PASS_FLG	SYNC1_PASS interrupt flag
	4	R	PREAM_PASS_FLG	PREAM_PASS interrupt flag
	3	R	SYNC_PASS_FLG	SYNC_PASS interrupt flag
	2	R	ADDR_PASS_FLG	ADDR_PASS interrupt flag
	1	R	CRC_PASS_FLG	CRC_PASS interrupt flag

	0	R	PKT_DONE_FLG	PKT_DONE interrupt flag
CTL_REG_28 (0x1C)	7	R	RX_FIFO_FULL_FLG	Interrupt to indicate RX FIFO full 0 : invalid 1 : valid
	6	R	RX_FIFO_NMTY_FLG	Indicates interrupt flag of RX FIFO not full 0 : invalid 1 : valid
	5	R	RX_FIFO_TH_FLG	An interrupt indicating that RX FIFO unread content exceeds FIFO TH 0 : invalid 1 : valid
	3	R	RX_FIFO_OVF_FLG	Interrupt indicating RX FIFO overflow 0 : invalid 1 : valid
	2	R	TX_FIFO_FULL_FLG	An interrupt indicating that the TX FIFO is not full. 0 : invalid 1 : valid
	1	R	TX_FIFO_NMTY_FLG	An interrupt indicating that the TX FIFO is not full. 0 : invalid 1 : valid
	0	R	TX_FIFO_TH_FLG	An interrupt indicating that RX FIFO unread content exceeds FIFO TH. 0 : invalid 1 : valid
CTL_REG_29 (0x1D)	4	W	ANT_LOCK_CLR	ANT_LOCK interrupt clearing 0: no action. 1: clearing.
	3	W	OP_CMD_FAILED_CLR	OP_CMD_FAILED interrupt clearing 0: no action. 1: clearing.
	2	W	RSSI_COLL_CLR	RSSI_COLL interrupt clearing 0: no action. 1: clearing.
	1	W	PKT_ERR_CLR	PKT_ERR interrupt clearing 0: no action. 1: clearing.
	0	W	LBD_STATUS_CLR	LBD_STATUS interrupt clearing 0: no action. 1: clearing.
CTL_REG_30 (0x1E)	4	R	ANT_LOCK_FLAG	ANT_LOCK interrupt flag
	3	R	OP_CMD_FAILED_FLG	OP_CMD_FAILED interrupt flag
	2	R	RSSI_COLL_FLG	RSSI_COLL interrupt flag
	1	R	PKT_ERR_FLG	PKT_ERR interrupt flag
	0	R	LBD_STATUS_FLG	LBD_STATUS interrupt flag

CTL_REG_31 (0x1F)	7	W	API_DONE_CLR	API_DONE interrupt clearing 0: no action. 1: clearing.
	6	W	CCA_STATUS_CLR	CCA_STATUS interrupt clearing 0: no action. 1: clearing.
	5	W	CSMA_DONE_CLR	CSMA_DONE interrupt clearing 0: no action. 1: clearing.
	4	W	TX_DC_DONE_CLR	TX_DC_DONE interrupt clearing 0: no action. 1: clearing.
	3	W	ACK_RECV_FAILED_CLR	ACK_RECV_FAILED interrupt clearing 0: no action. 1: clearing.
	2	W	TX_RESEND_DONE_CLR	TX_RESEND_DONE interrupt clearing 0: no action. 1: clearing.
	1	W	NACK_RECV_CLR	NACK_RECV interrupt clearing 0: no action. 1: clearing.
	0	W	SEQ_MATCH_CLR	SEQ_MATCH interrupt clearing 0: no action. 1: clearing.
CTL_REG_32 (0x20)	7	R	API_DONE_FLG	API_DONE interrupt flag
	6	R	CCA_STATUS_FLG	CCA_STATUS interrupt flag
	5	R	CSMA_DONE_FLG	CSMA_DONE interrupt flag
	4	R	TX_DC_DONE_FLG	TX_DC_DONE interrupt flag
	3	R	ACK_RECV_FAILED_FLG	ACK_RECV_FAILED interrupt flag
	2	R	TX_RESEND_DONE_FLG	TX_RESEND_DONE interrupt flag
	1	R	NACK_RECV_FLG	NACK_RECV interrupt flag
	0	R	SEQ_MATCH_FLG	SEQ_MATCH interrupt flag

The interrupt mapping table is shown in the following table. As the mapping of INT 1 and INT 2 is the same, let's take INT1 as an example.

Table 32. CMT2310A Interrupt Mapping Table

Name	INT1_SEL	Description	Clearing Method
INT_MIX	000000	Mixed interrupts, INT_MIX is valid if any of the following interrupts are valid	Auto/By MCU
ANT_LOCK	000001	The antenna locking interruption complete when the antenna diversity function operates.	By MCU
RSSI_PJD_VALID	000010	Valid interrupt of RSSI and PJD combination	Auto
PREAM_PASS	000011	Indicate interruption of successfully receive the Preamble	By MCU
SYNC_PASS	000100	Indicate interruption of successfully receive the Sync Word.	By MCU
ADDR_PASS	000101	Indicate interruption of successfully receive the Addr interrupt.	By MCU

CRC_PASS	000110	Indicate interruption of successfully receive and pass the CRC verification.	By MCU
PKT_OK	000111	Indicate interruption of successfully receive a complete and correct packet	By MCU
PKT_DONE	001000	There will be 4 cases when the packet received is finished: 1. Completely receive the entire right data packet 2. Manchester decode error, decoding circuit restart automatically 3. NODE ID receive error, decoding circuit restart automatically 4. Signal conflict, decoding circuit cannot restart automatically and waits for the MCU processing.	By MCU
SLEEP_TMO	001001	Indicate interruption of SLEEP counter timed out.	By MCU
RX_TMO	001010	Indicate interruption of RX counter timed out.	By MCU
RX_FIFO_NMTY	001011	Indicate interruption of RX FIFO not empty.	Auto
RX_FIFO_TH	001100	Indicate interruption of RX FIFO unread content exceeding FIFO TH.	Auto
RX_FIFO_FULL	001101	Indicate interruption of RX FIFO full	Auto
RX_FIFO_WBYTE	001110	Indicate interruption of writing a BYTE into RX FIFO, which is a pulse.	Auto
RX_FIFO_OVF	001111	Indicate interruption of RX FIFO overflow	Auto
TX_DONE	010000	Indicate interruption of TX complete.	By MCU
TX_FIFO_NMTY	010001	Indicate interruption of TX FIFO not empty	Auto
TX_FIFO_TH	010010	Indicate interruption of TX FIFO unread content exceeding FIFO TH.	Auto
TX_FIFO_FULL	010011	Indicate interruption of TX FIFO full	Auto
STATE_IS_READY	010100	Indicate that it is interruption under the READY state.	Auto
STATE_IS_FS	010101	Indicate that it is interruption under the state of RFS and TFS.	Auto
STATE_IS_RX	010110	Indicate that it is interruption under the RX state.	Auto
STATE_IS_TX	010111	Indicate that it is interruption under the TX state.	Auto
LBD_STATUS	011000	Indicate interruption of low voltage detection valid (VDD is lower than the setting TH)	By MCU
API_CMD_FAILED	011001	Interruption of API command execute error	By MCU
API_DONE	011010	Interruption of API command complete	By MCU
TX_DC_DONE	011011	Interruption of Duty Cycle reaching to maximum number under TX mode.	By MCU
ACK_RECV_FAILED	011100	Interruption of ACK received failed	By MCU
TX_RESEND_DONE	011111	Interruption of repeatedly transmission reaching to maximum number.	By MCU
NACK_RECV	011110	Interruption of receiving indication of NACK	By MCU
SEQ_MATCH	011111	Interruption of sequence matching successfully	By MCU
CSMA_DONE	100000	Interruption of CSMA operating to maximum number.	By MCU
CCA_STATUS	100001	Interruption of channel sensing state	By MCU

Interrupts are valid at 1 by default, but all interrupts can be valid at 0 by setting the register bit INT_POLAR to 1.

Take INT1 as an example, the following figure shows control and selection of two interrupt sources with different properties. The control and mapping of INT1 and INT2 is the same.

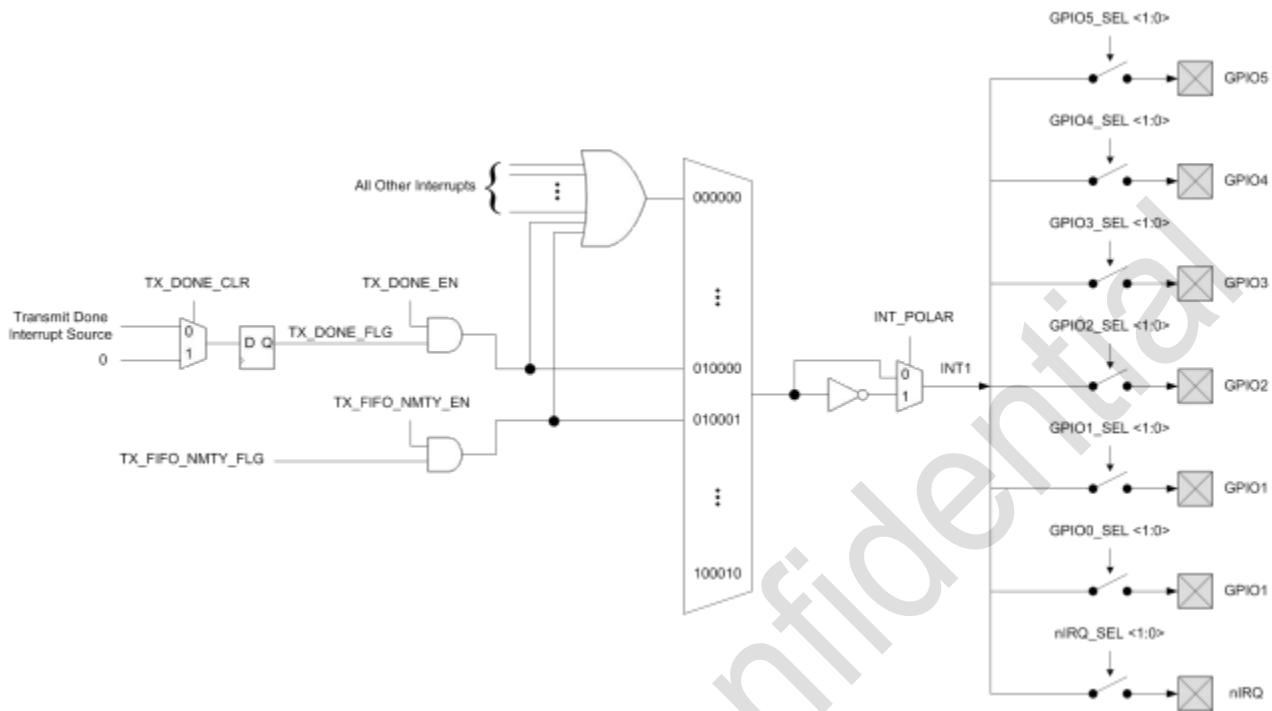


Figure 29. CMT2310A INT1 Interrupt Mapping

For interrupt sources that require MCU clearing, each comes with an EN enable and a CLR clear bit. For example, the interrupt source for TX_DONE will only be generated if TX_DONE_EN bit is set to 1, and when this interrupt is generated,

TX_DONE_CLR will be reset only if it is set to 1. When setting the CLR bit, the MCU only needs to set it to 1, and then does not need to set it back to 0, because the CLR bit will be cleared automatically within the chip after the corresponding interrupt is cleared.

PKT_OK and PKT_DONE interrupts are fundamentally different. PKT_OK is one of the cases of PKT_DONE, which means that the packet is received completely. However, there will be 3 unexpected cases when successfully detected SYNC WORD at practical application:

1. If the packet contains DEST ADDR (or DEST ADDR / SRC ADDR), a detection error may occur. Then the PKT_ERR_FLG flag will be set, the chip will stop receiving packets, and automatically restart the decoder, waiting for the next SYNC WORD. However, PKT_OK will not be generated in this case
2. If Manchester decoding is enabled, it is possible that a decoding error will occur and PKT_ERR_FLG flag bit will be set, the chip will stop receiving packets, and automatically restart the decoder and wait for the next SYNC WORD. However, PKT_OK will not be generated in this case
3. If signal conflict detection is enabled (detailed usage will be introduced later), signal conflict may be found, which will lead to errors in the contents of the packets received later. At this time, the COL_ERR_FLG flag bit will be set, and the

chip will continue to receive packets until completion, and the decoder will not restart automatically. In this case, PKT_OK will still be generated although the received data is wrong.

Regardless of which situation happens, it needs to tell the outside MCU, otherwise MCU will keep waiting for the interruption and interacting with chip failed. So, interruption signal of PKT_DONE = PKT_OK | PKT_ERR_FLG | COL_ERR_FLG occurs for notifying MCU. Once the MCU received the interruption it can find out the corresponding signal bit and keep processing.

Alternatively, the MCU could simply wait for PKT_OK and then check for CRC signal bit (if any), however, be careful to work with SYNC_OK interrupts. For example, if the first packet fails to decode, the decoder will immediately re-receive the next packet. For the MCU, two SYNC_OK interrupts will occur in a row. If the MCU fails to handle the SYNC_OK interrupt, it will assume that the second SYNC_OK interrupt is PKT_OK, and the processing is wrong.

In conclusion, for the reason that codec interrupt will encounter some changeable situation and users understand differently, it is recommended that the application has to be possibly simple and put the stability in the first place. Moreover, it is best to have a timeout mechanism. If MCU and chip interact failed (loss of contact), crash will occur.

See Appendix 2 for examples of CMT2310 GPIO output interrupt configuration code.

The use of TX_DONE interrupt:

When the user uses Packet mode for transmission, CMT2310A will automatically exit TX state after transmitting the current packets (or N packets set by the user). TX_DONE interrupts occurred when the TX state is safely exited.

Therefore, users only need to set TX_EXIT_STATE in advance to set the chip status after automatically exit TX state, and it will switch to either SLEEP/READY/TFS state. After that, MCU only needs to detect TX_DONE interruption to determine

whether to exit TX, and there is no need to exit TX by sending state switch command.

3.3 Antenna TX/RX Switching Control

The registers which control external antenna TX/RX switching are listed in the table below.

Table 33. TX/RX Switching Control Related Register

Register Name	Bits	R/W	Bit Name	Function Description
CTL_REG_22 (0x16)	1	RW	TRX_SWT_INV	Two output channel of TX/RX antenna on/off control: 0: not reverse. 1: reverse.
	0	RW	TRX_SWT_EN	Set GPIO0 and GPIO1 as the control output of antenna TX/RX on/off. The priority of this one is higher than that of GPIO0_SEL and GPIO1_SEL.

The chip can output a group of signals (with 2 different signals) on GPIO0 and GPIO1 to control RX/TX switching of external antenna.

4. Appendix

4.1 Appendix 1: Sample code for FIFO read/write

Sample code for FIFO read/write operation sub-function.

```
/*! *****  
* @name      cmt2310a_fifo_rd_wr_sel  
* @desc      read or write fifo(only valid by fifo merge enable)  
* @param     rw    0: read fifo, 1: write fifo  
* *****/  
void cmt2310a_fifo_rd_wr_sel(u8 rw)  
{  
    cmt2310a_set_reg_bits(CMT2310A_CTL_REG_19, (rw << BIT0), M_FIFO_RD_WR_SEL);  
}
```

4.2 Appendix 2: Sample code for GPIO output interrupt configuration function

```

void rf_config(void)
{
    /* Config registers */
    cmt2310a_config_page_regs(0, g_cmt2310a_page0, CMT2310A_PAGE0_SIZE); /* config page 0 */
    cmt2310a_config_page_regs(1, g_cmt2310a_page1, CMT2310A_PAGE1_SIZE); /* config page 1 */
#ifdef CMT2310A_AUTO_RX_HOP_ENABLE /* cmt2310a_param.h exported by rfpdk */
    cmt2310a_select_reg_page(2);
    for(u8 idx = 0; idx < freq_times_val; idx++)
    {
        cmt2310a_write_reg(idx, g_cmt2310a_page2[idx]); /* config page 2 */
    }
    cmt2310a_select_reg_page(0);
    cmt2310a_write_reg(CMT2310A_CTL_REG_12, freq_space_val);
    cmt2310a_write_reg(CMT2310A_CTL_REG_13, freq_times_val);
    cmt2310a_write_reg(CMT2310A_CTL_REG_14, freq_switch_state_val);
#endif

    cmt2310a_go_ready();
    cmt2310a_delay_ms(2);

```

```

/* do ir calibration */
cmt2310a_api_command_and_wait(0x01);
cmt2310a_api_command_and_wait(0x01);

/* Config GPIOs */
cmt2310a_config_gpio0(GPIO0_SEL_INT1); /* INT1 -> GPIO0 */
cmt2310a_config_gpio1(GPIO1_SEL_INT2); /* INT2 -> GPIO1 */

/* Config interrupt */
cmt2310a_config_interrupt(
    INT_SRC_TX_DONE, /* Config TX_DONE -> INT1 */
    INT_SRC_PKT_DONE /* Config PKT_DONE -> INT2 */
);

```

```
/* config interrupt mode */
cmt2310a_config_interrupt_mode(INT1_TYPE_SEL_MODE1, INT2_TYPE_SEL_MODE1);

/* config interrupt polar */
cmt2310a_set_interrupt_polar(0,0); /* INT1: active-high, INT2: active-high */

/* enable interrupt source */
cmt2310a_enable_interrupt_source_0(
    M_TX_DONE_EN      |
    M_PREAM_PASS_EN   |
    M_SYNC_PASS_EN     |
    M_ADDR_PASS_EN     |
    M_CRC_PASS_EN      |
    M_PKT_DONE_EN
);

/* Use a single 256-byte FIFO for either Tx or Rx */
//cmt2310a_enable_fifo_merge(1);

//cmt2310a_set_fifo_threshold(16);

/* save fifo at sleep state or not */
//cmt2310a_fifo_sleep_save(0); /* 0: save, 1: no save */

/* select pa mode, 0: single, 1: difference */
cmt2310a_select_pa_mode(0);
}
```

5. Revise History

Table 34 Revise History Records

Version No.	Chapter	Description	Date
0.5	All	Initial version	2020-09-17
0.6	2.5/2.7/2.10/2.11	2.5: Update the detection way of Address field and reallocate the definition of Address field. 2.7: Update the description of Payload total length and DATA length under variable packet structure. Update the description of Length Byte. 2.10: Add the description of Wi-sun data packet. 2.11: Update the description of Tx Packet Number configuration.	2021-09-17
0.6A	All	Review	2022-01-09
0.6C	All	Review	2022-08-03
0.7	All	Review	2022-08-17
0.8	All	Disable Direct Tx mode and all the GPIO related TX DIN is disabled.	2023-04-26

6. Contacts

Shenzhen Hope Microelectronics Co., Ltd. Shenzhen Branch

Address: 30th floor, 8th Building, C Zone, Vanke Cloud City, Xili Sub-district, Nanshan, Shenzhen, GD, P.R. China

Post code: 518055

Tel: +86 - 755 - 82973805

Sales: sales@hoperf.com

Supports: support@hoperf.com

Website: www.hoperf.com

All rights reserved at Shenzhen Hope Microelectronics Co., Ltd.

The information furnished by HOPERF is believed to be accurate and reliable. However, HOPERF reserves rights to change, correct or modify products or documents at anytime without any notice. Without the written permission of the Company, no unit or individual shall copy part of or all of the contents of this document, and shall not disseminate it in any form. The content of this document will be updated irregularly due to product version upgrades or other reasons. HOPERF's products are not recommended to be applied to life-related equipment and systems, and HOPERF shall not be liable for any losses caused by the equipment or operation failure of the system in using the device.

HOPERF and some other related **HOPERF** is registered trademark of Shenzhen Hope Microelectronics Co., Ltd. However, all other registered names mentioned above are the property of their respective owners.